

Transformer

Trình bày: Nguyễn Hồng Bửu Long



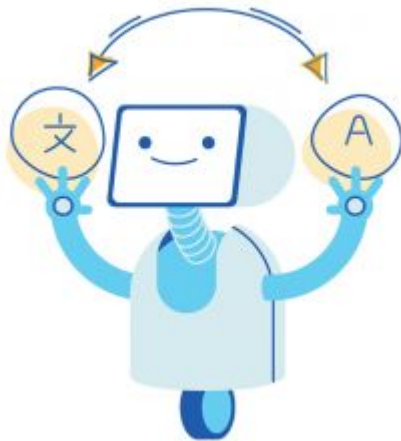
fit@hcmus

KHOA CÔNG NGHỆ THÔNG TIN
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN

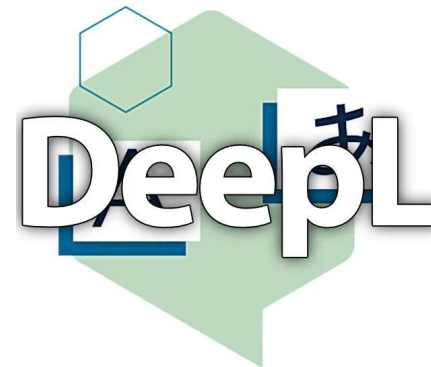
- ❑ Machine Translation
- ❑ Transformer

MACHINE TRANSLATION (MT)

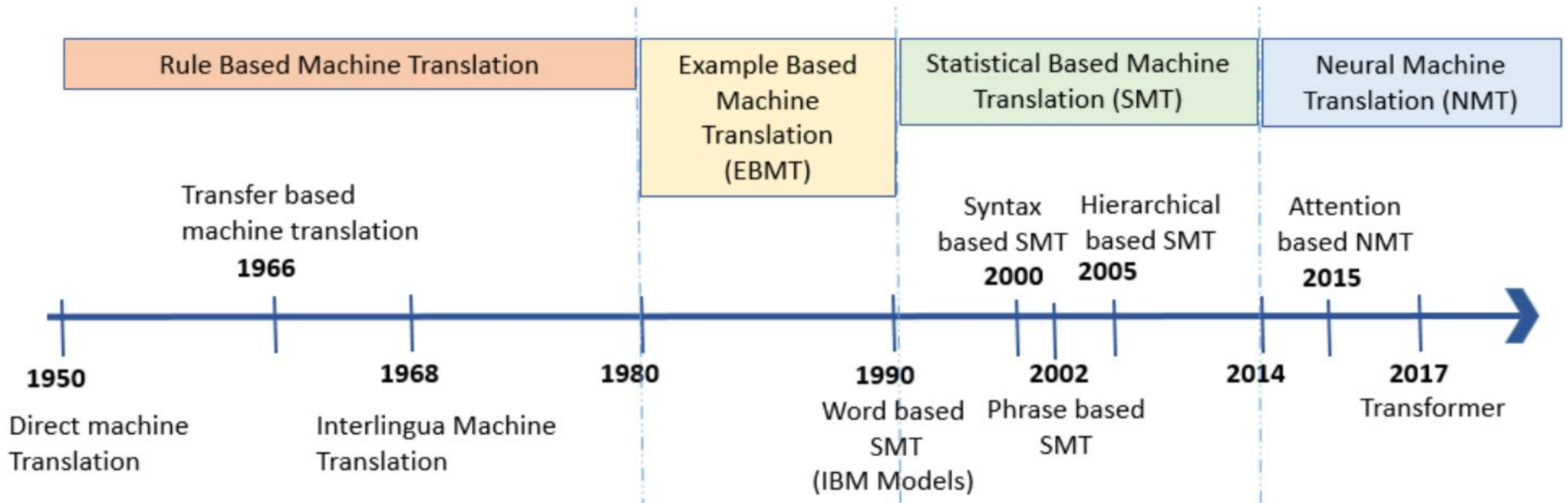
MT-Indoduction



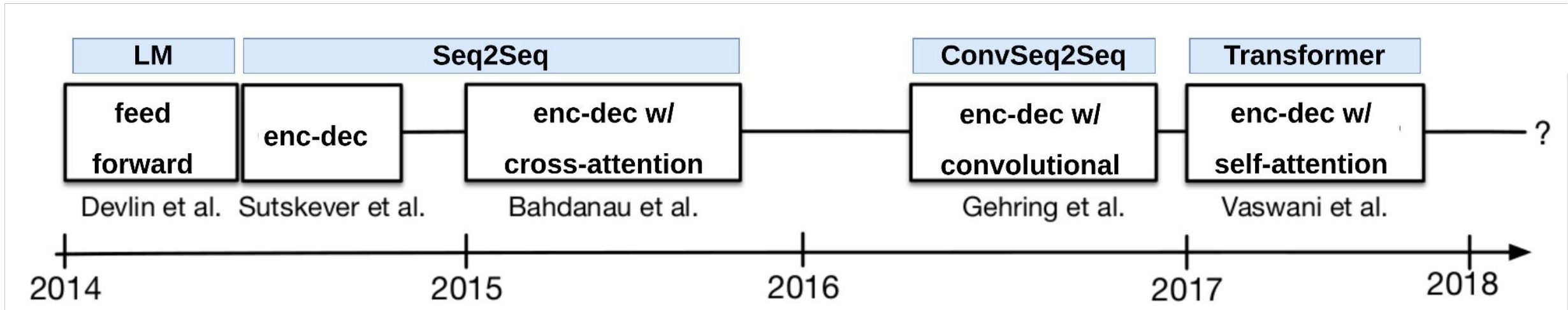
Commercial Systems



Evolution of MT systems



Evolution of NMT systems



Machine Translation System definition

Human Translation

$$y^* = \arg \max_y p(y|x)$$

The “probability” is intuitive and is given by a human translator’s expertise

Machine Translation

$$y' = \arg \max_y p(y|x, \theta)$$

model parameters

Questions we need to answer

- modeling

How does the model for $p(y|x, \theta)$ look like?

- learning

How to find θ ?

- search

How to find the argmax?

NMT Architecture

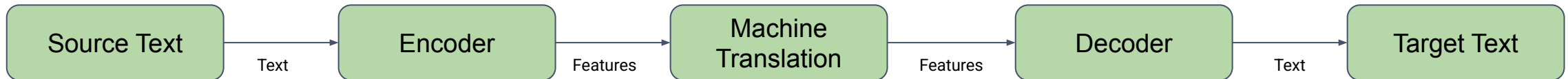
Shannon-Weaver's Model of Communication

(A Mathematical Theory of Communication, Shannon et al., 1948)



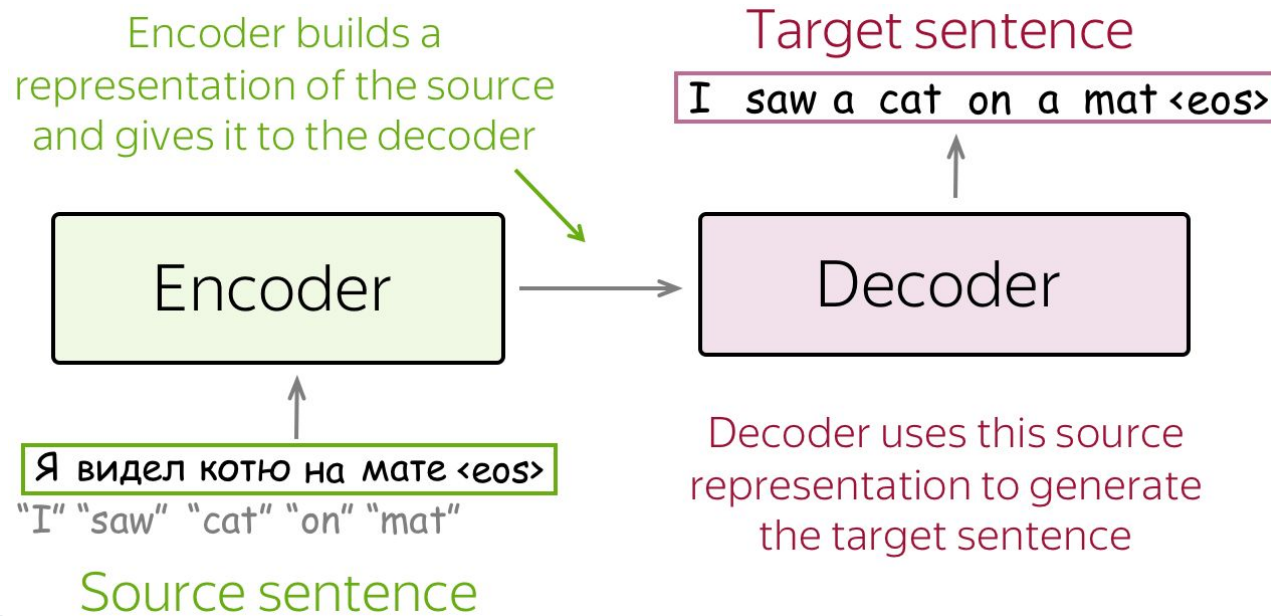
Machine Translation

(A Statistical Approach to Machine Translation, Brown et al., 1989)



Encoder-Decoder Framework

Encoder-decoder is the standard modeling paradigm for sequence-to-sequence tasks:



Encoder-Decoder Framework (cnt.)

Language Models: $P(y_1, y_2, \dots, y_n) = \prod_{t=1}^n p(y_t | y_{<t})$

Conditional

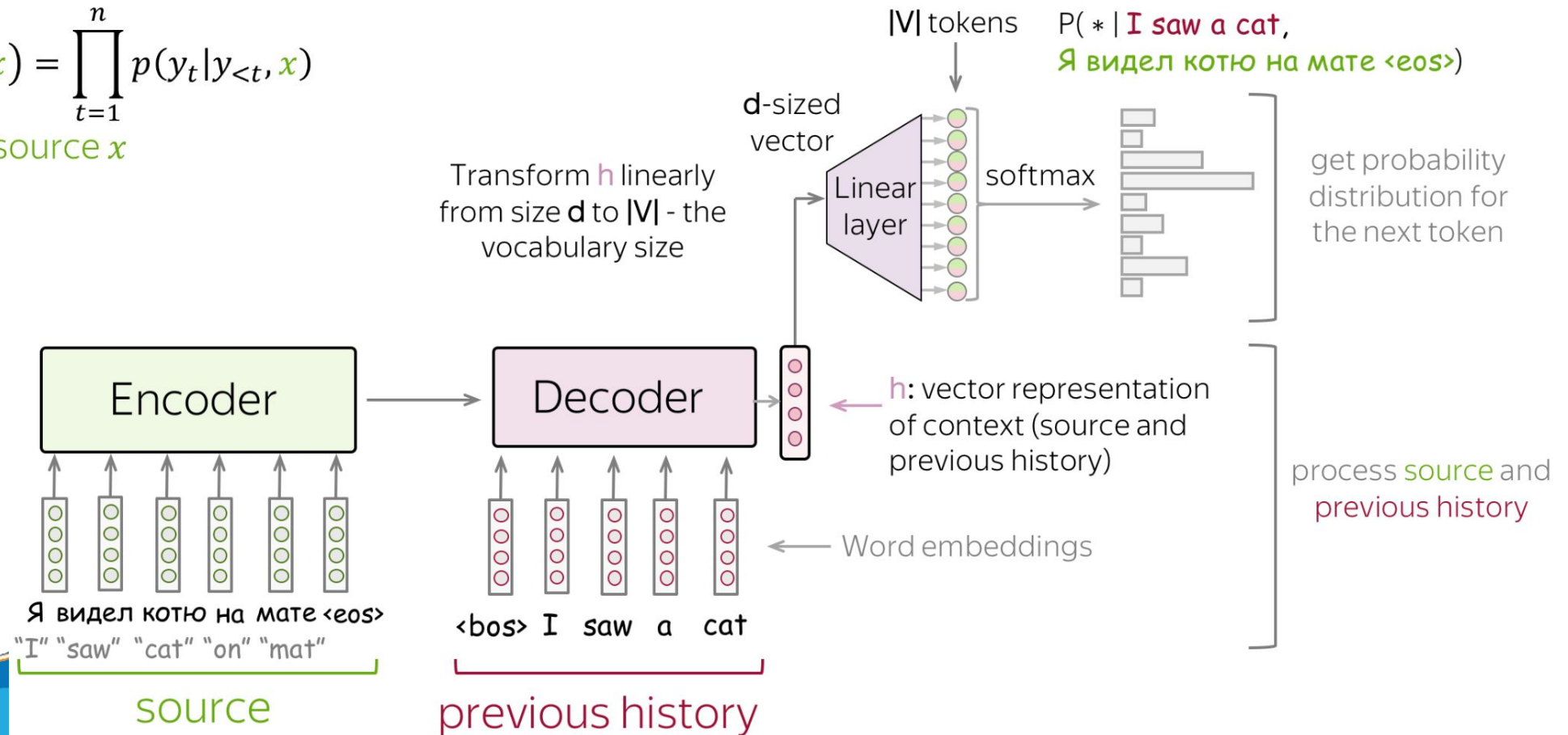
Language Models: $P(y_1, y_2, \dots, y_n, | \overset{\uparrow}{x}) = \prod_{t=1}^n p(y_t | y_{<t}, x)$

condition on source x

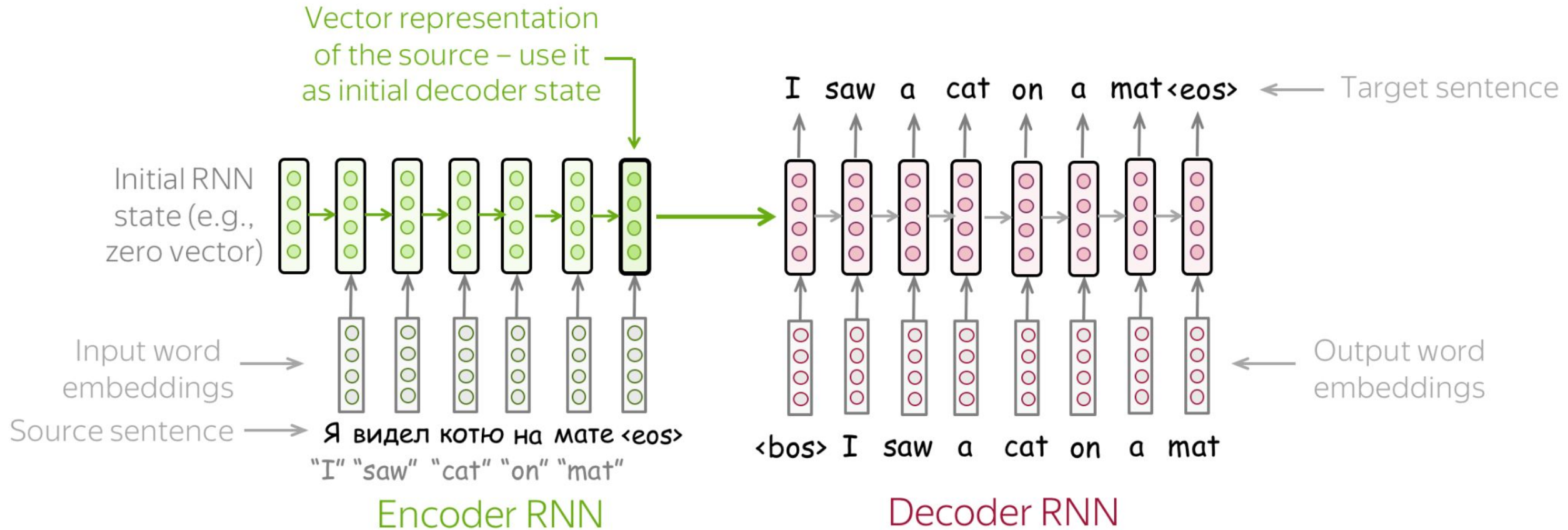
Encoder-Decoder Framework (cnt.)

$$P(y_1, y_2, \dots, y_n, |x) = \prod_{t=1}^n p(y_t | y_{<t}, x)$$

condition on source x

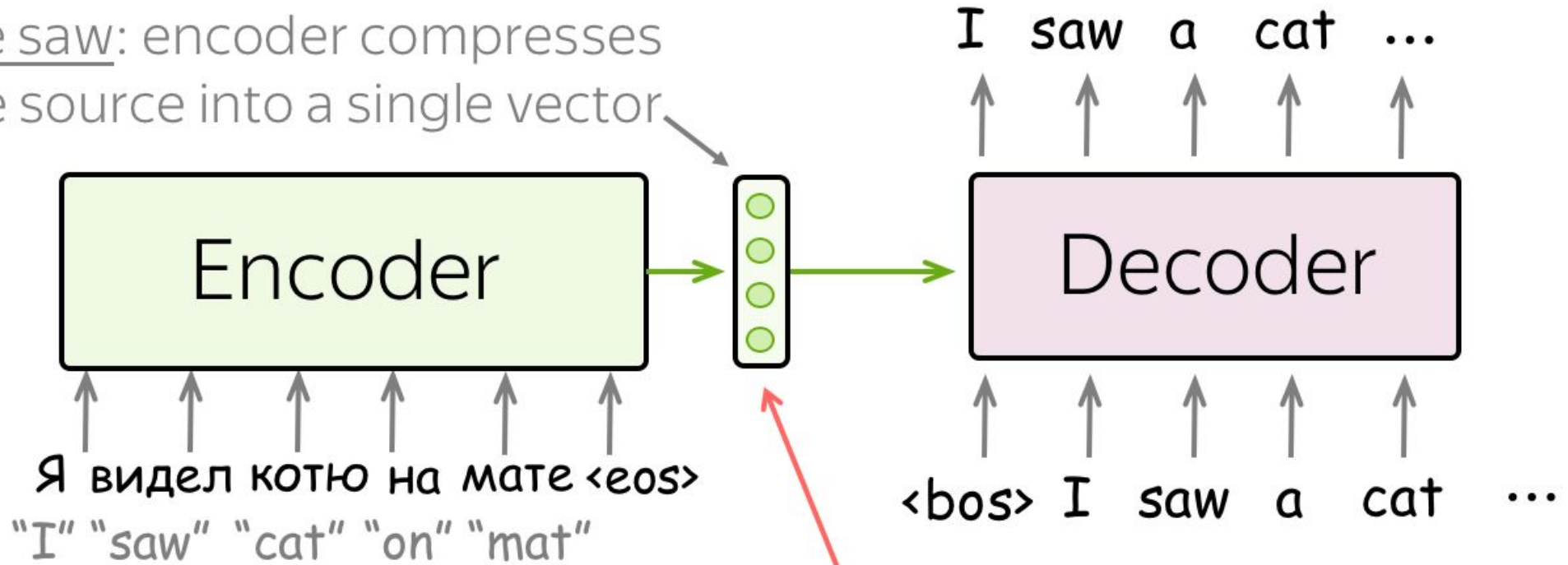


The Simplest Model: Two RNNs for Encoder and Decoder



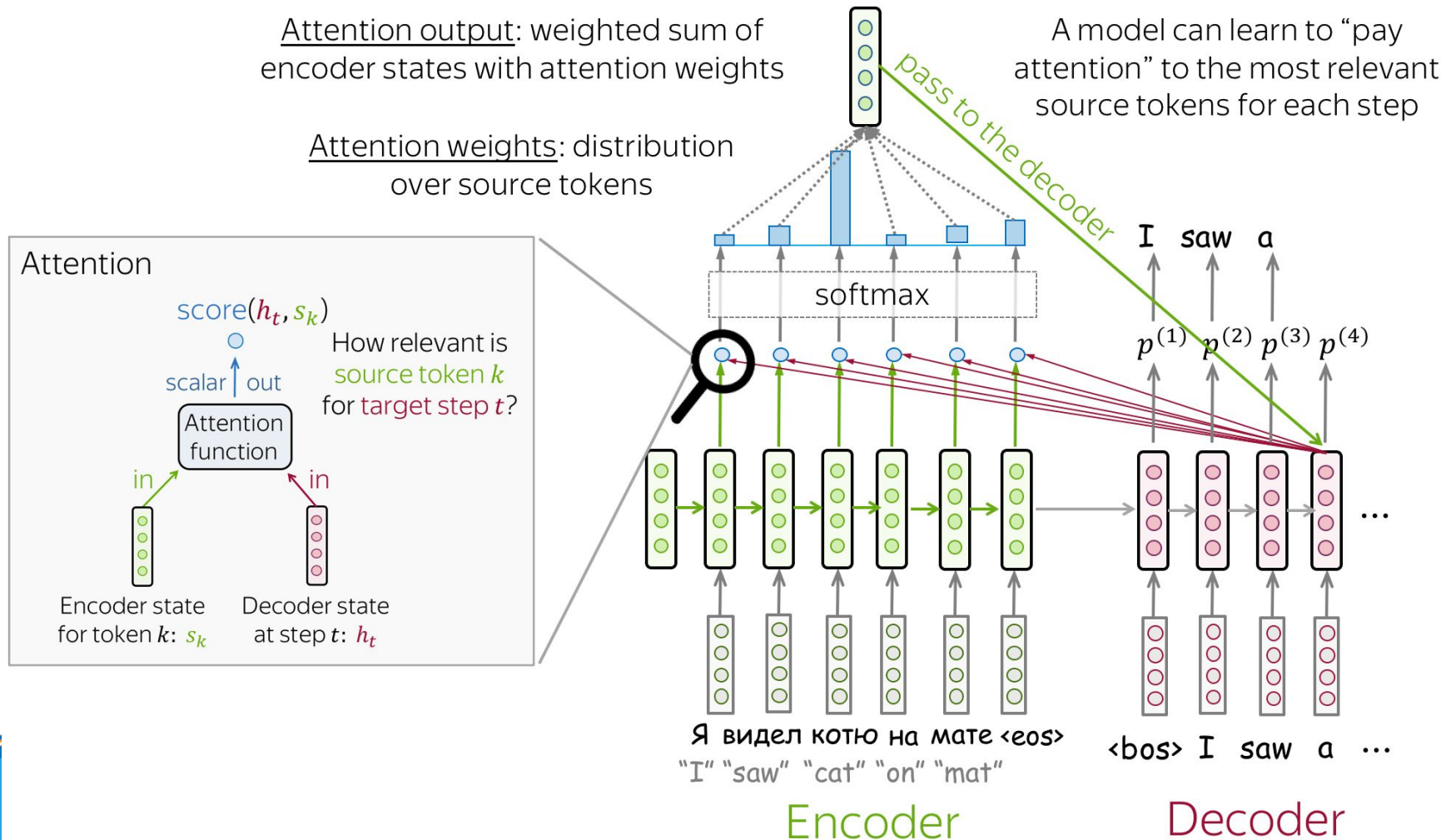
The Problem of Fixed Encoder Representation

We saw: encoder compresses the source into a single vector

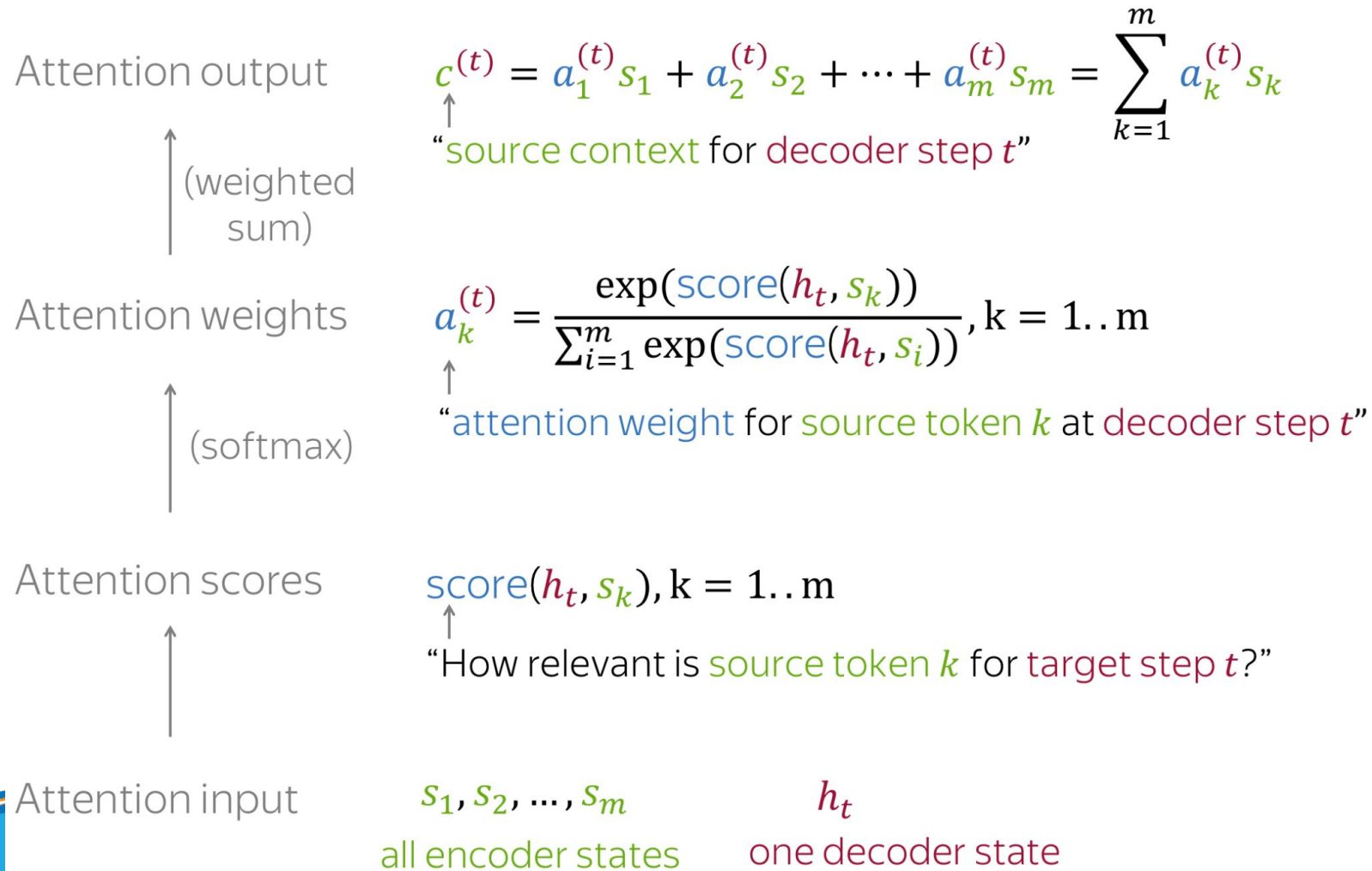


Problem: this is a bottleneck!

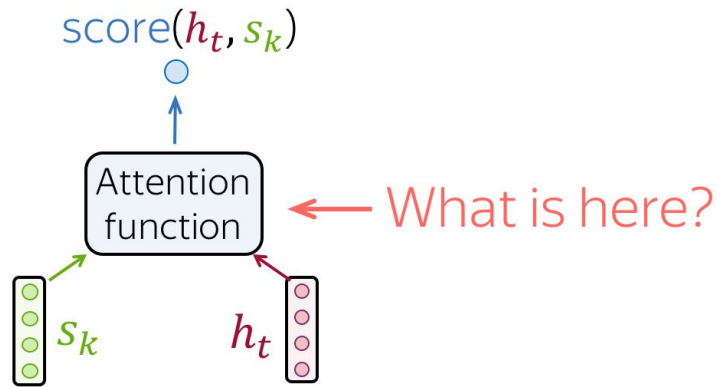
Attention: A High-Level View



Attention: Computation Scheme



Attention: Compute Attention Score



Dot-product

$$h_t^T \times s_k$$

$$\text{score}(h_t, s_k) = h_t^T s_k$$

Bilinear

$$h_t^T \times [W] \times s_k$$

$$\text{score}(h_t, s_k) = h_t^T W s_k$$

Multi-Layer Perceptron

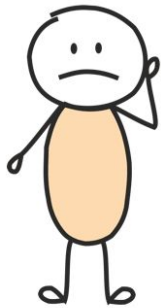
$$w_2^T \times \tanh \left[W_1 \times \begin{bmatrix} h_t \\ s_k \end{bmatrix} \right]$$

$$\text{score}(h_t, s_k) = w_2^T \cdot \tanh(W_1 [h_t, s_k])$$

Problems with RNN-based NMT

I arrived at the **bank** after crossing thestreet? ...river?

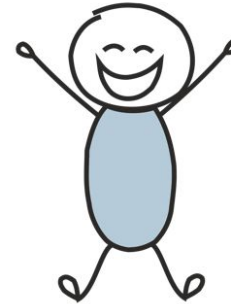
What does **bank** mean in this sentence?



I've no idea: let's wait until I read the end

RNNs

$O(N)$ steps to process a sentence with length N



I don't need to wait - I see all words at once!

Transformer

Constant number of steps to process any sentence

TRANSFORMER: Attention is All You Need

Vaswani et al., 2017

Transformer: Attention is All You Need

	Seq2seq without attention	Seq2seq with attention	Transformer
processing within encoder	RNN/CNN	RNN/CNN	attention
processing within decoder	RNN/CNN	RNN/CNN	attention
decoder-encoder interaction	static fixed- sized vector	attention	attention

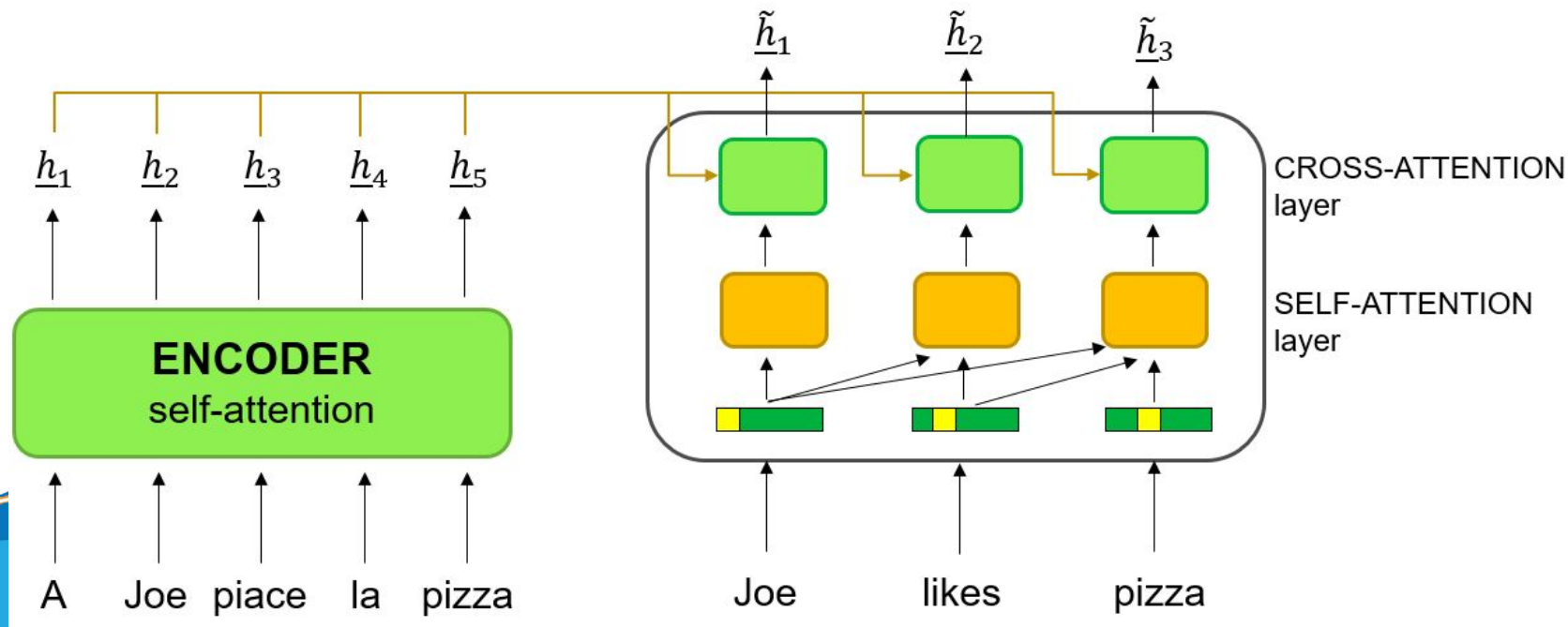
Self-Attention: the "Look at Each Other" Part

Decoder-encoder attention is looking

- **from:** one current decoder state
- **at:** all encoder states

Self-attention is looking

- **from:** each state from a set of states
- **at:** all other states in the same set



Query, Key, and Value in Self-Attention

query - asking for information;

key - saying that it has some information;

value - giving the information.

Each vector receives three representations ("roles")

$$[W_Q] \times \begin{bmatrix} \bullet \\ \bullet \\ \bullet \end{bmatrix} = \begin{bmatrix} \bullet \\ \bullet \end{bmatrix}$$

Query: vector from which the attention is looking

"Hey there, do you have this information?"

$$[W_K] \times \begin{bmatrix} \bullet \\ \bullet \\ \bullet \end{bmatrix} = \begin{bmatrix} \bullet \\ \bullet \end{bmatrix}$$

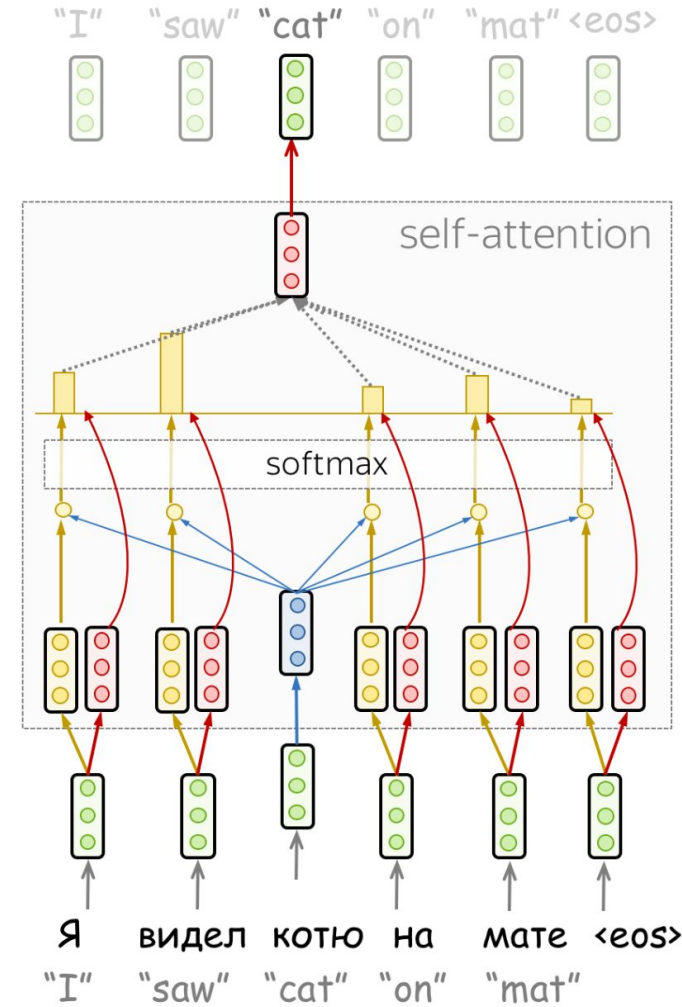
Key: vector at which the query looks to compute weights

"Hi, I have this information - give me a large weight!"

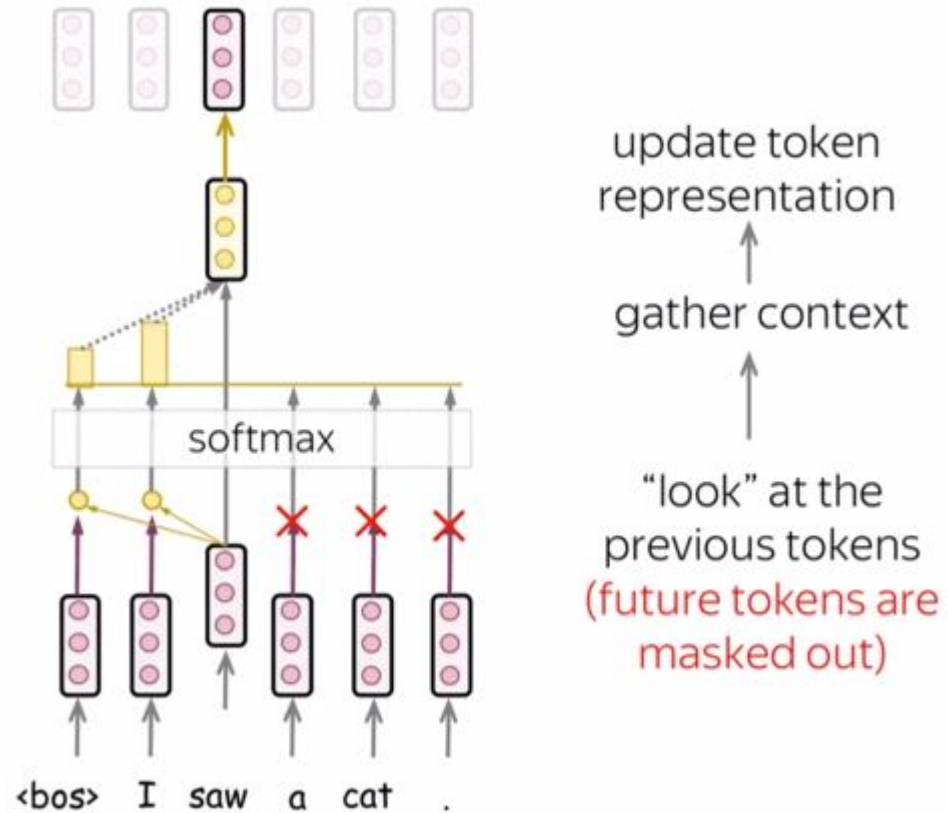
$$[W_V] \times \begin{bmatrix} \bullet \\ \bullet \\ \bullet \end{bmatrix} = \begin{bmatrix} \bullet \\ \bullet \end{bmatrix}$$

Value: their weighted sum is attention output

"Here's the information I have!"

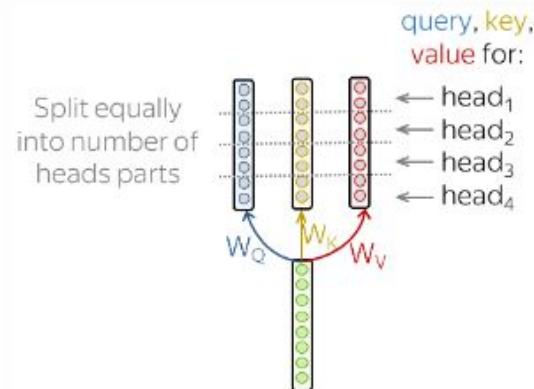


Masked Self-Attention: "Don't Look Ahead" for the Decoder



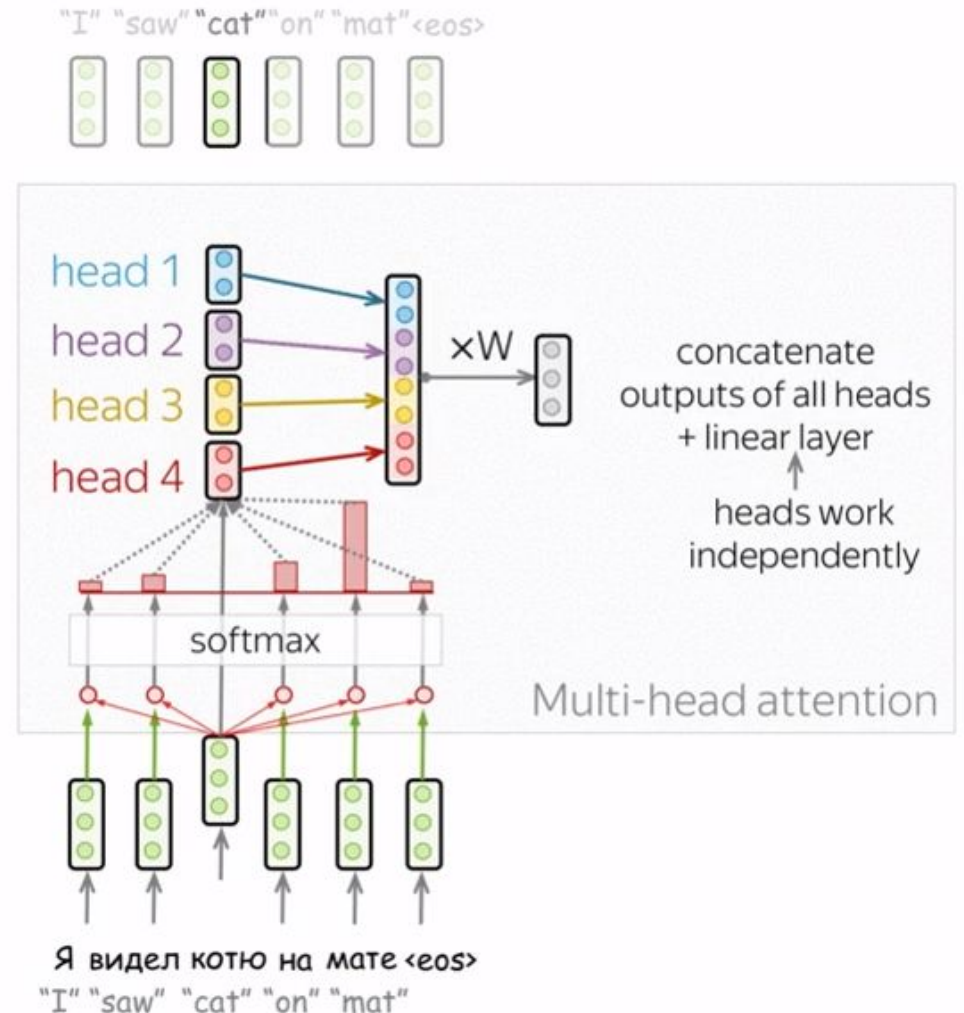
Multi-Head Attention: Independently Focus on Different Things

Formally, this is implemented as several attention mechanisms whose results are combined:

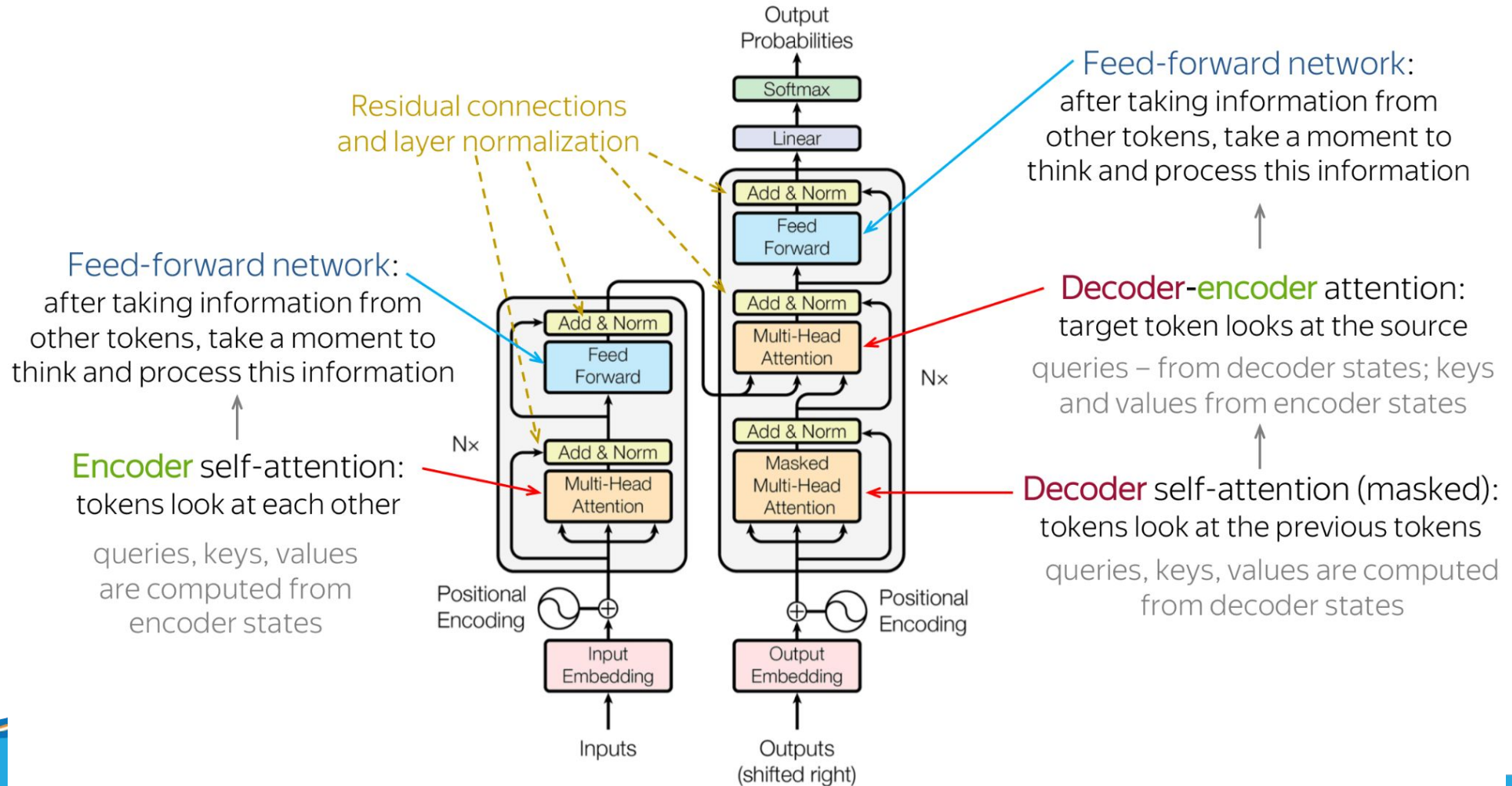


$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_n) W_O,$$

$$\text{head}_i = \text{Attention}(QW_Q^i, KW_K^i, VW_V^i)$$



Transformer: Model Architecture



Positional encoding

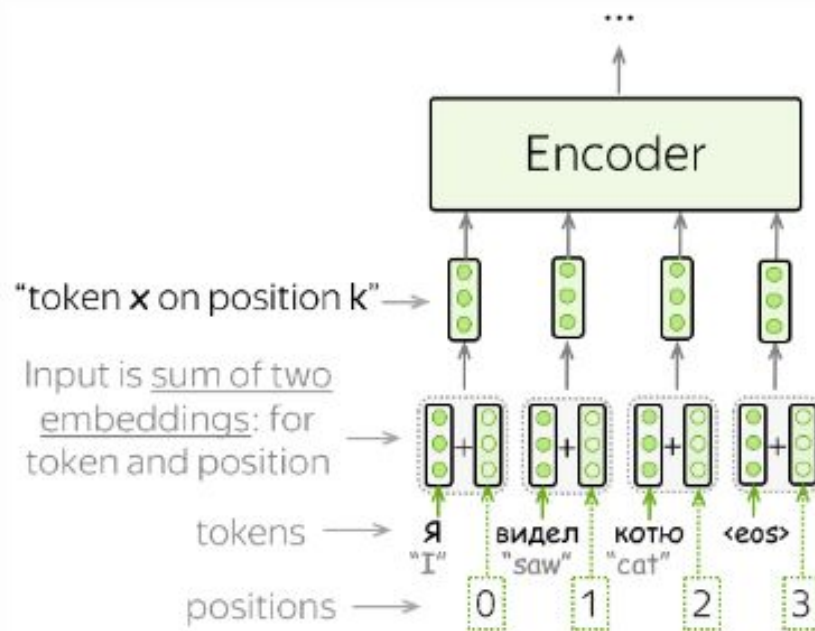
Note that since Transformer does not contain recurrence or convolution, it does not know the order of input tokens. Therefore, we have to let the model know the positions of the tokens explicitly. For this, we have two sets of embeddings: for tokens (as we always do) and for positions (the new ones needed for this model). Then input representation of a token is the sum of two embeddings: token and positional.

The positional embeddings can be learned, but the authors found that having fixed ones does not hurt the quality. The fixed positional encodings used in the Transformer are:

$$PE_{pos,2i} = \sin(pos/10000^{2i/d_{model}}),$$

$$PE_{pos,2i+1} = \cos(pos/10000^{2i/d_{model}}),$$

where pos is position and i is the vector dimension. Each dimension of the positional encoding corresponds to a sinusoid, and the wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$.

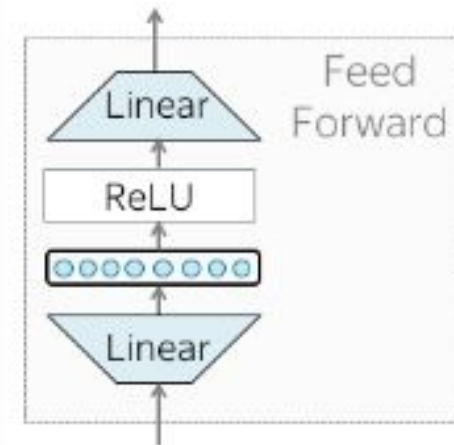


Feed-forward blocks

In addition to attention, each layer has a feed-forward network block: two linear layers with ReLU non-linearity between them:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2.$$

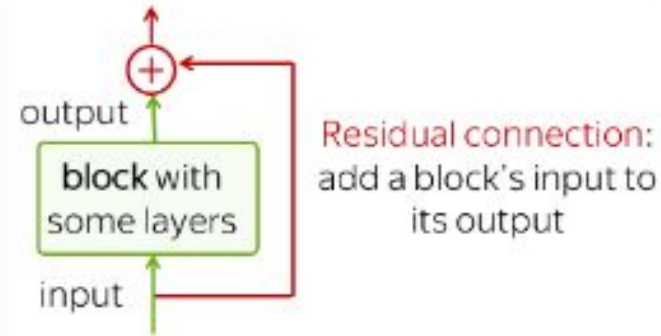
After looking at other tokens via an attention mechanism, a model uses an FFN block to process this new information (attention - "look at other tokens and gather information", FFN - "take a moment to think and process this information").



Residual connections

We already saw residual connections when talking about **convolutional language models**. Residual connections are very simple (add a block's input to its output), but at the same time are very useful: they ease the gradient flow through a network and allow stacking a lot of layers.

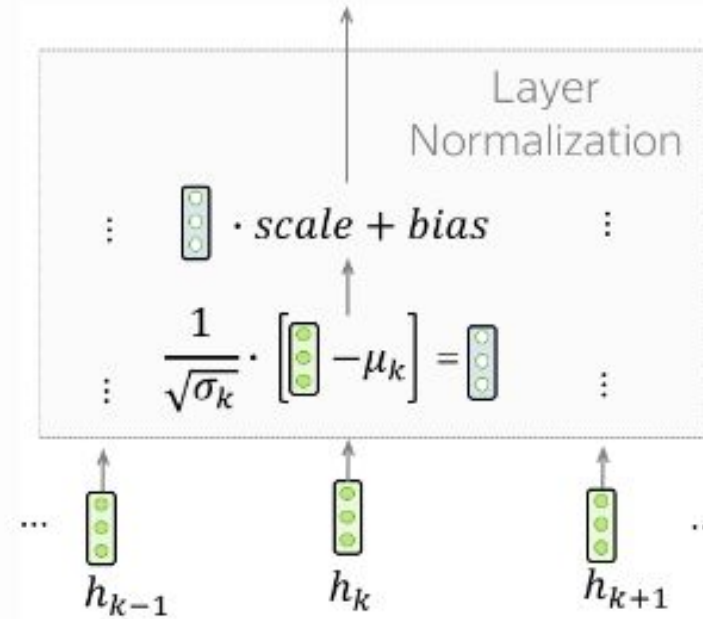
In the Transformer, residual connections are used after each attention and FFN block. On the illustration above, residuals are shown as arrows coming around a block to the yellow "Add & Norm" layer. In the "Add & Norm" part, the "Add" part stands for the residual connection.



Layer Normalization

The "Norm" part in the "Add & Norm" layer denotes [Layer Normalization](#). It independently normalizes vector representation of each example in batch - this is done to control "flow" to the next layer. Layer normalization improves convergence stability and sometimes even quality.

In the Transformer, you have to normalize vector representation of each token. Additionally, here LayerNorm has trainable parameters, *scale* and *bias*, which are used after normalization to rescale layer's outputs (or the next layer's inputs). Note that μ_k and σ_k are evaluated for each example, but *scale* and *bias* are the same - these are layer parameters.



Encoder

1. Input Embeddings with Positional Encoding:

$$X' = X + \text{PositionalEncoding}(X)$$

2. Multi-Head Self-Attention:

$$Q_i = X_l W_i^Q$$

$$K_i = X_l W_i^K$$

$$V_i = X_l W_i^V$$

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i)$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$Z_l = \text{LayerNorm}(X_l + \text{MultiHead}(Q, K, V))$$

3. Feed-Forward Neural Network:

$$\text{FFN}(Z) = \text{ReLU}(ZW_1 + b_1)W_2 + b_2$$

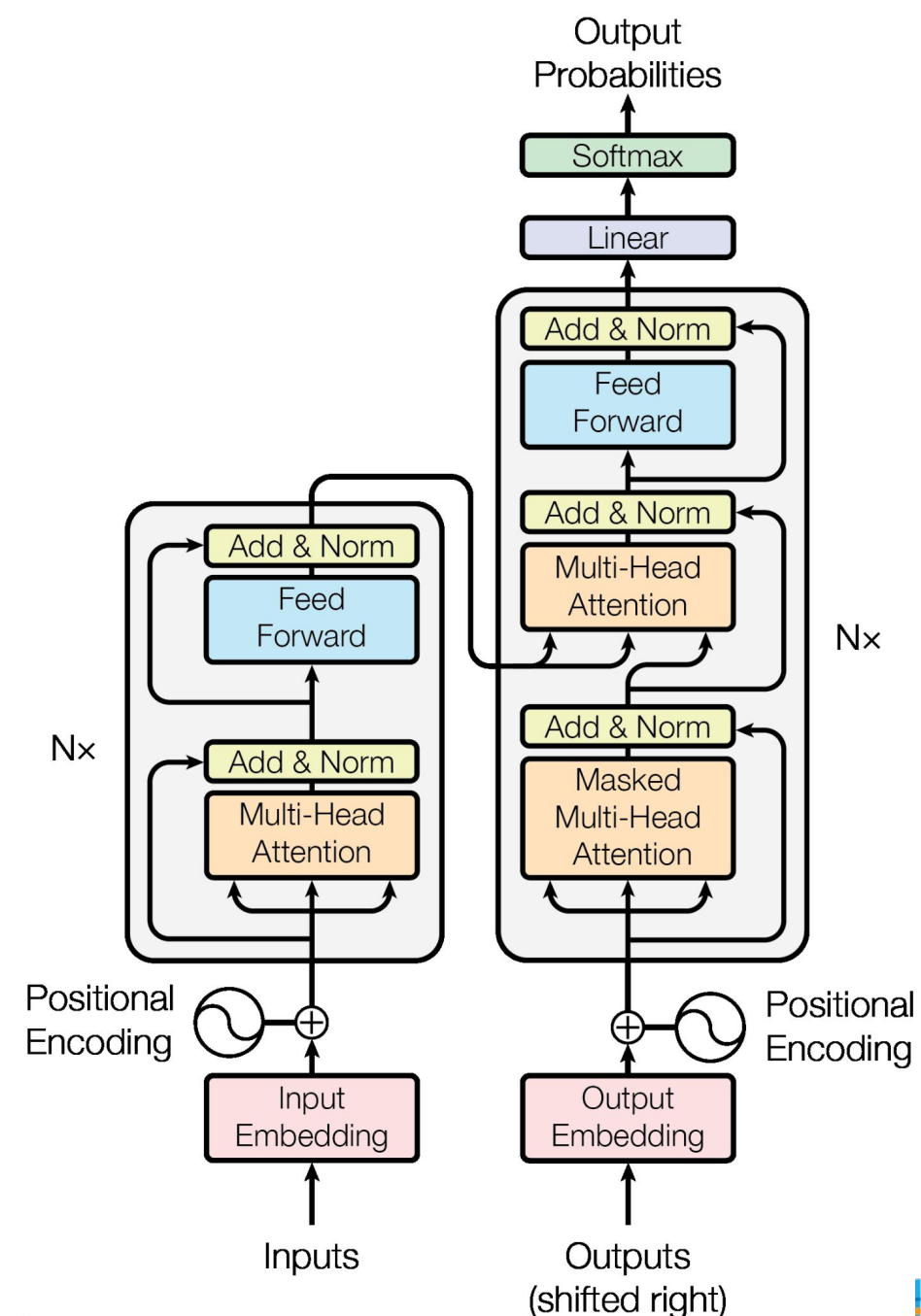
$$X_{l+1} = \text{LayerNorm}(Z_l + \text{FFN}(Z_l))$$

4. Stacking Encoder Layers:

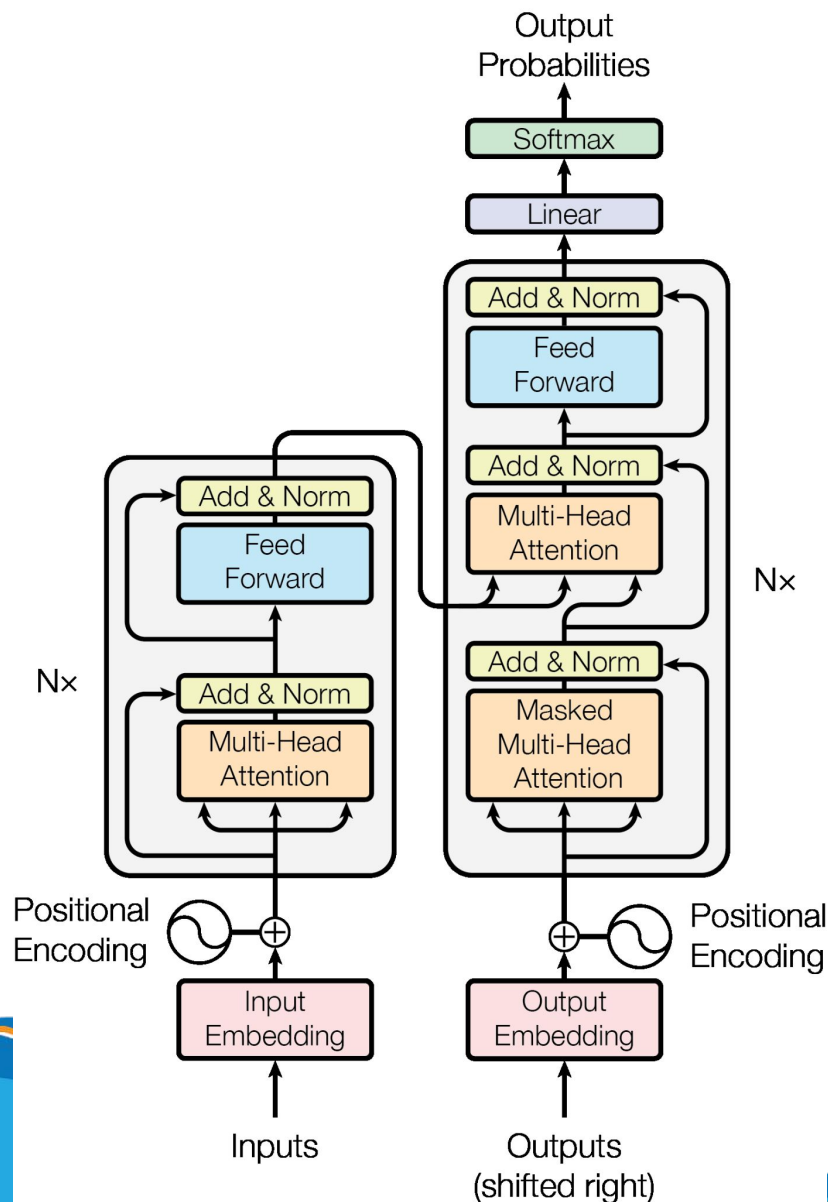
$$X_{L+1} = \text{EncoderLayer}(X_L)$$

5. Final Encoder Output:

$$\text{EncoderOutput} = X_{L+1}$$



Decoder



1. Input Embeddings with Positional Encoding:

$$Y' = Y + \text{PositionalEncoding}(Y)$$

2. Masked Multi-Head Self-Attention:

$$Q_i = Y_l W_i^Q$$

$$K_i = Y_l W_i^K$$

$$V_i = Y_l W_i^V$$

$$\text{head}_i = \text{MaskedAttention}(Q_i, K_i, V_i)$$

$$\text{MaskedMultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$Z_l = \text{LayerNorm}(Y_l + \text{MaskedMultiHead}(Q, K, V))$$

3. Multi-Head Attention over Encoder Output:

$$Q_i = Z_l W_i^Q$$

$$K_i = \text{EncoderOutput} W_i^K$$

$$V_i = \text{EncoderOutput} W_i^V$$

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

$$Z'_l = \text{LayerNorm}(Z_l + \text{MultiHead}(Q, K, V))$$

4. Feed-Forward Neural Network:

$$\text{FFN}(Z') = \text{ReLU}(Z' W_1 + b_1) W_2 + b_2$$

$$Y_{l+1} = \text{LayerNorm}(Z'_l + \text{FFN}(Z'_l))$$

5. Stacking Decoder Layers:

$$Y_{L+1} = \text{DecoderLayer}(Y_L)$$

6. Final Decoder Output:

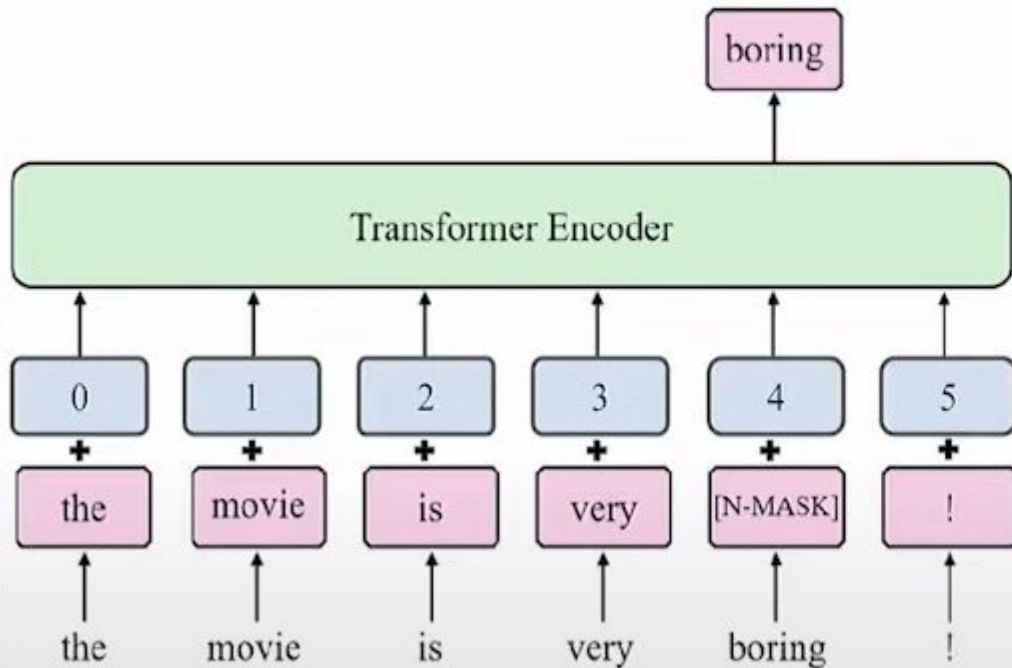
$$\text{DecoderOutput} = Y_{L+1}$$

Machine Translation Applications

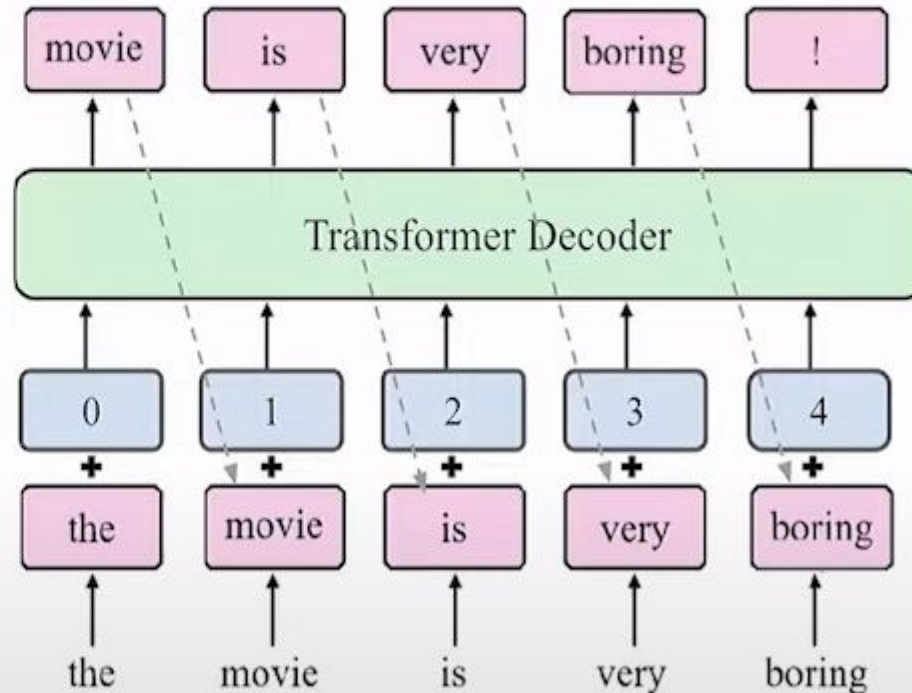
- ❖ Natural Language (NL) Translation:
 - NL -> NL: Text Translation;
 - Question -> Answer: Question Answering;
 - Input -> Response: Chatbot.
- ❖ Programming Language (PL) Translation:
 - NL -> PL: Code Generation;
 - PL -> NL: Code Description;
 - PL -> PL: Code Translation.

Transformer in Pre-trained LMs

Encoder-Only (e.g., BERT, RoBERTa)
(Pre-training Objective : Masked LM)



Decoder-Only (e.g., GPT-2, GPT-3)
(Pre-training Objective : Causal LM)



APPENDIX

Training: The Cross-Entropy Loss

The standard loss function is the cross-entropy loss. Cross-entropy loss for the target distribution p^* and the predicted distribution p is

$$Loss(p^*, p) = -p^* \log(p) = -\sum_{i=1}^{|V|} p_i^* \log(p_i).$$

Since only one of p_i^* is non-zero (for the correct token y_t), we will get

$$Loss(p^*, p) = -\log(p_{y_t}) = -\log(p(y_t | y_{<t}, x)).$$

Source sequence:

Я видел котю на мате <eos>
"I" "saw" "cat" "on" "mat"

Target sequence:

I saw a cat on a mat <eos>

← one training example

← one step for this example

previous tokens we want the model to predict this

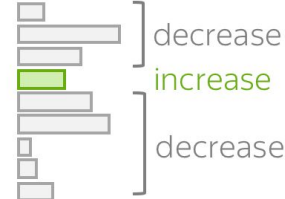
Model prediction: $p(* | \text{I saw a, Я ... <eos>})$



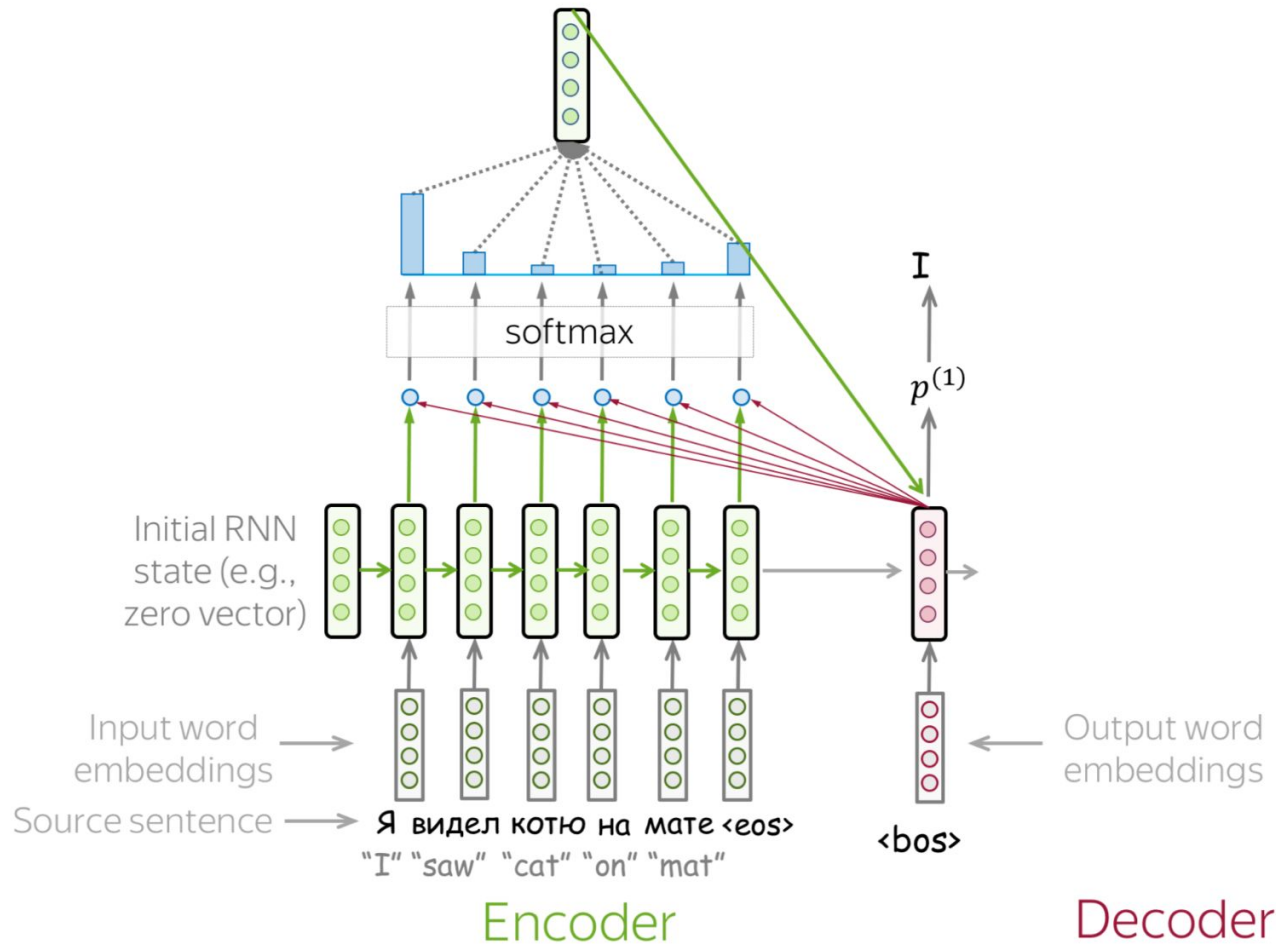
Target



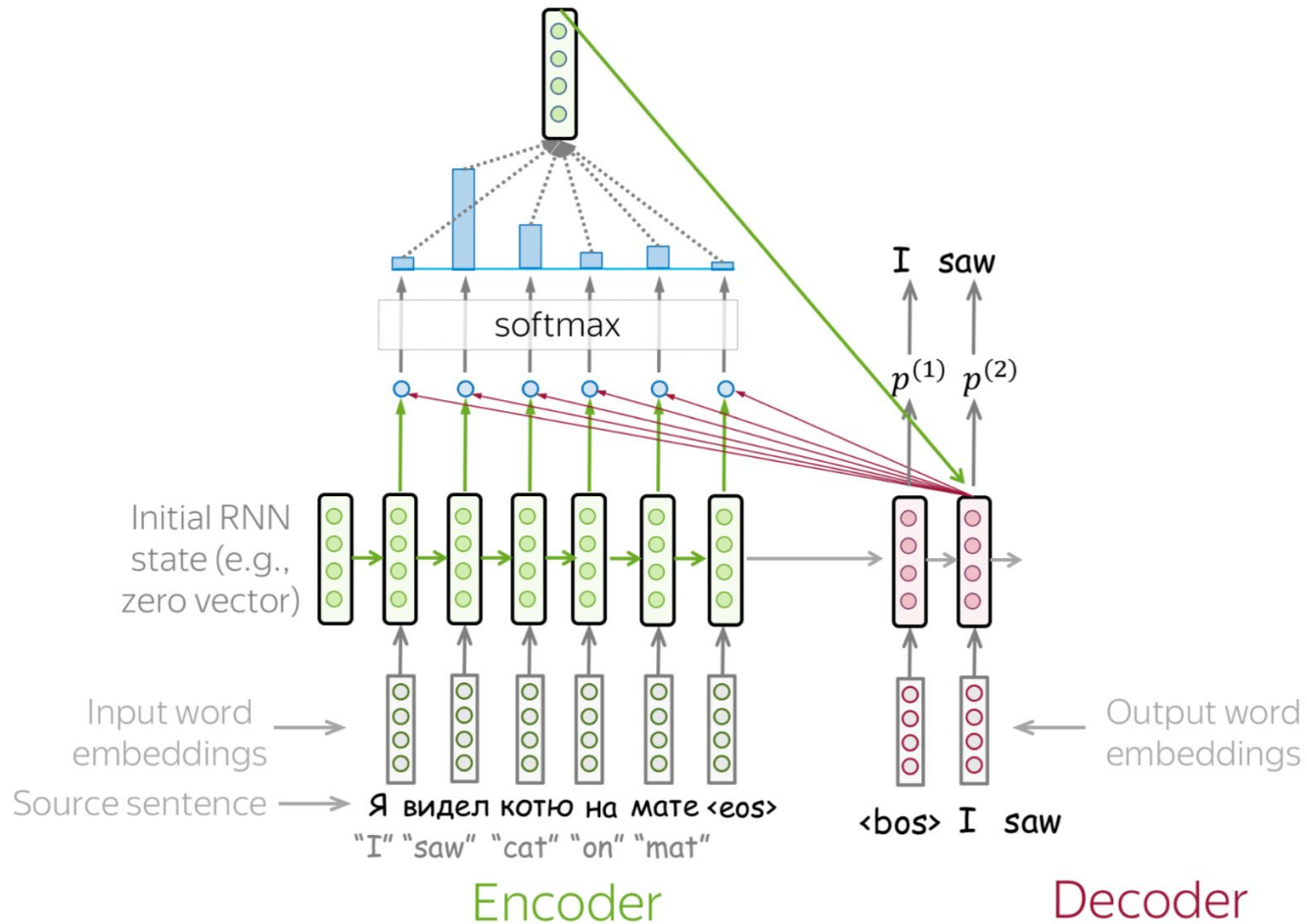
Loss = $-\log(p(\text{cat})) \rightarrow \min$



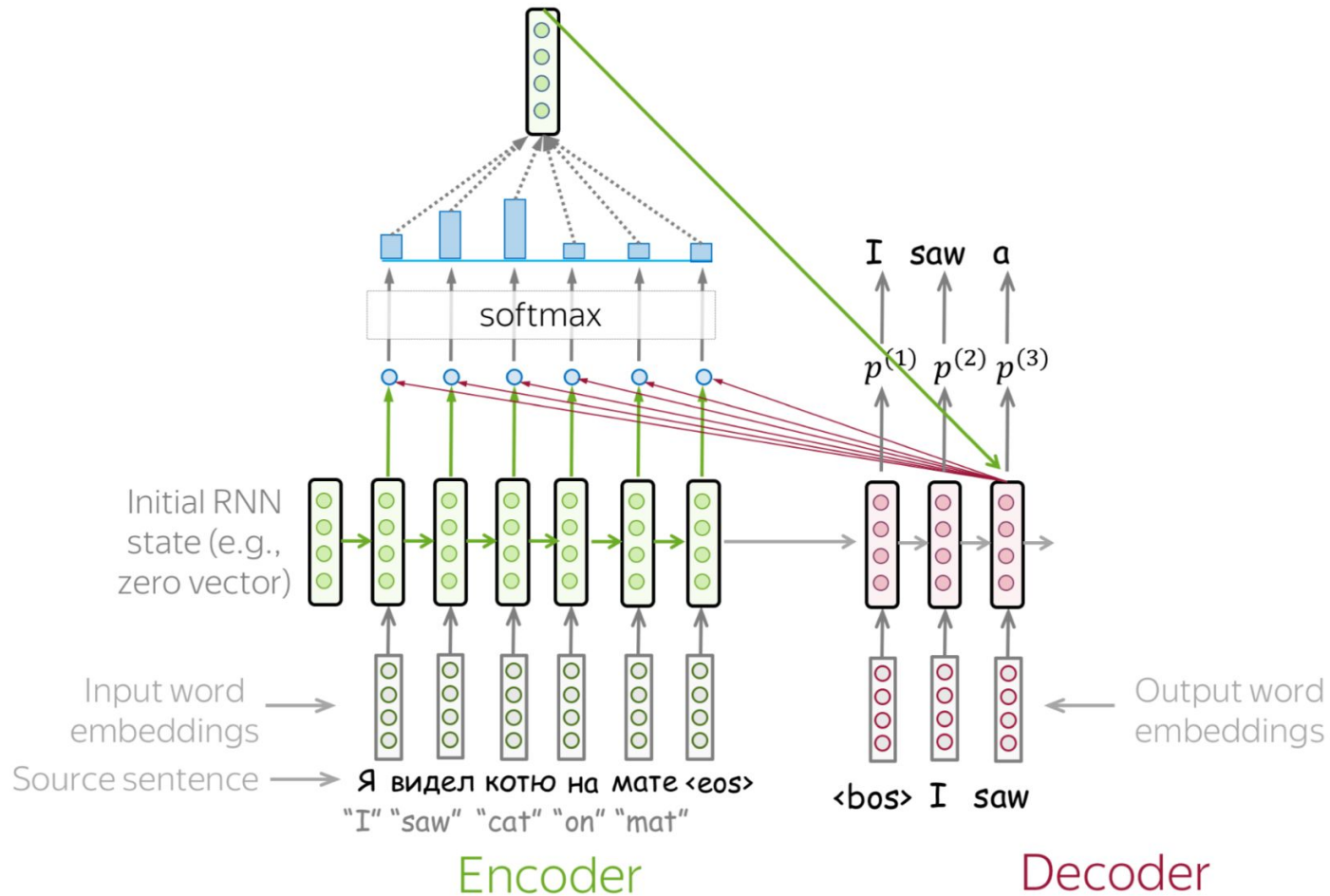
Attention: Steps



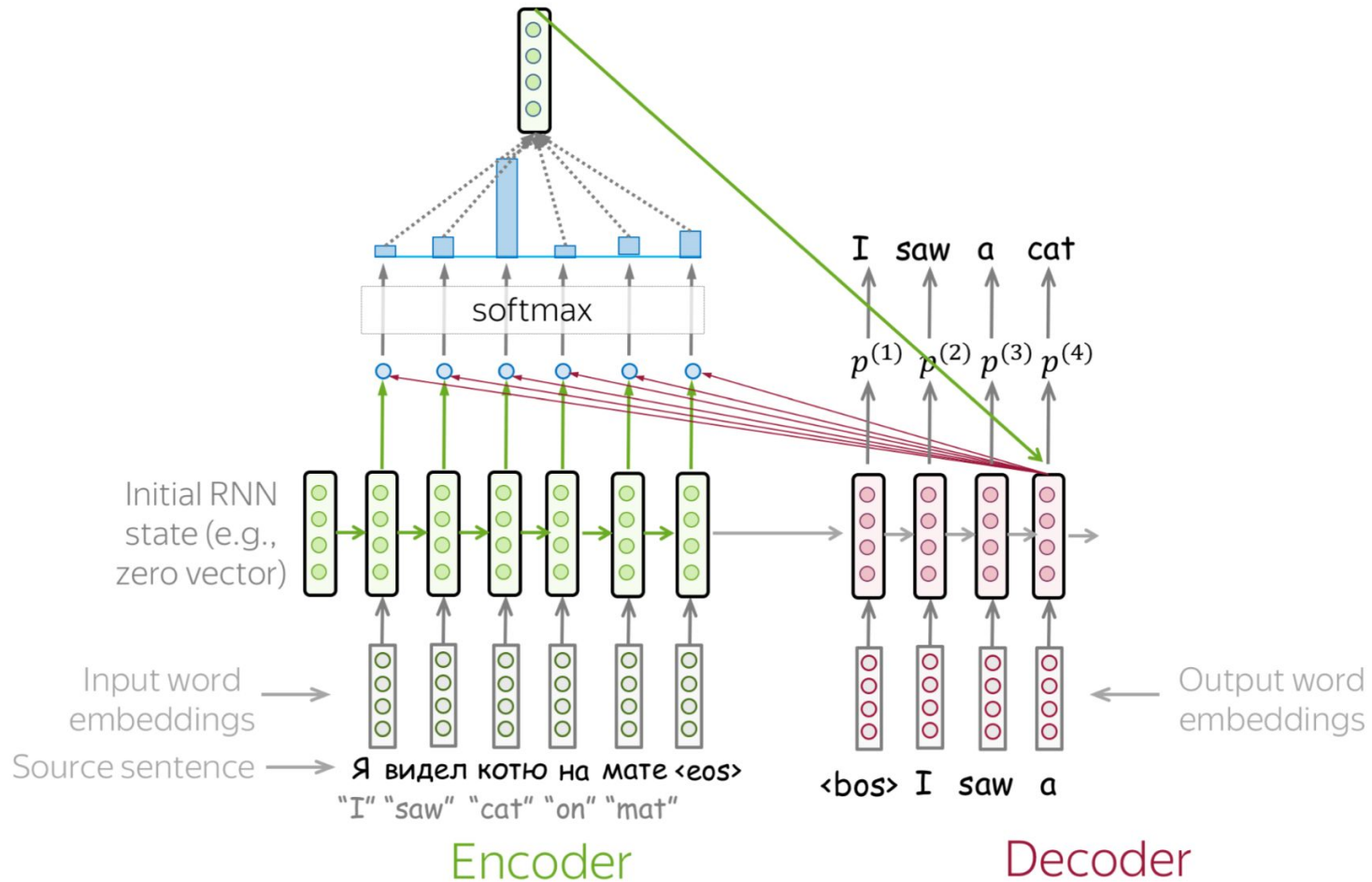
Attention: Steps



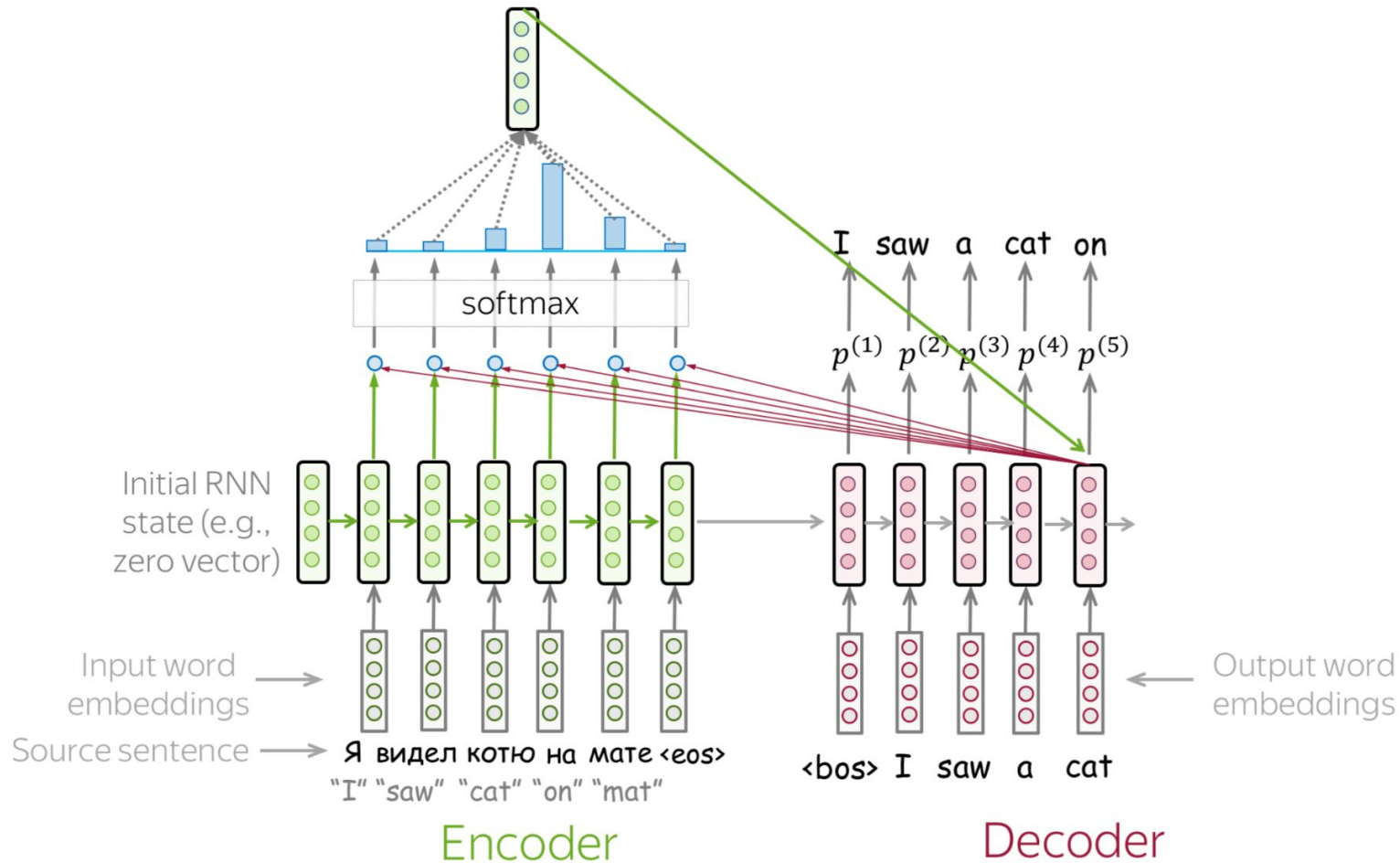
Attention: Steps



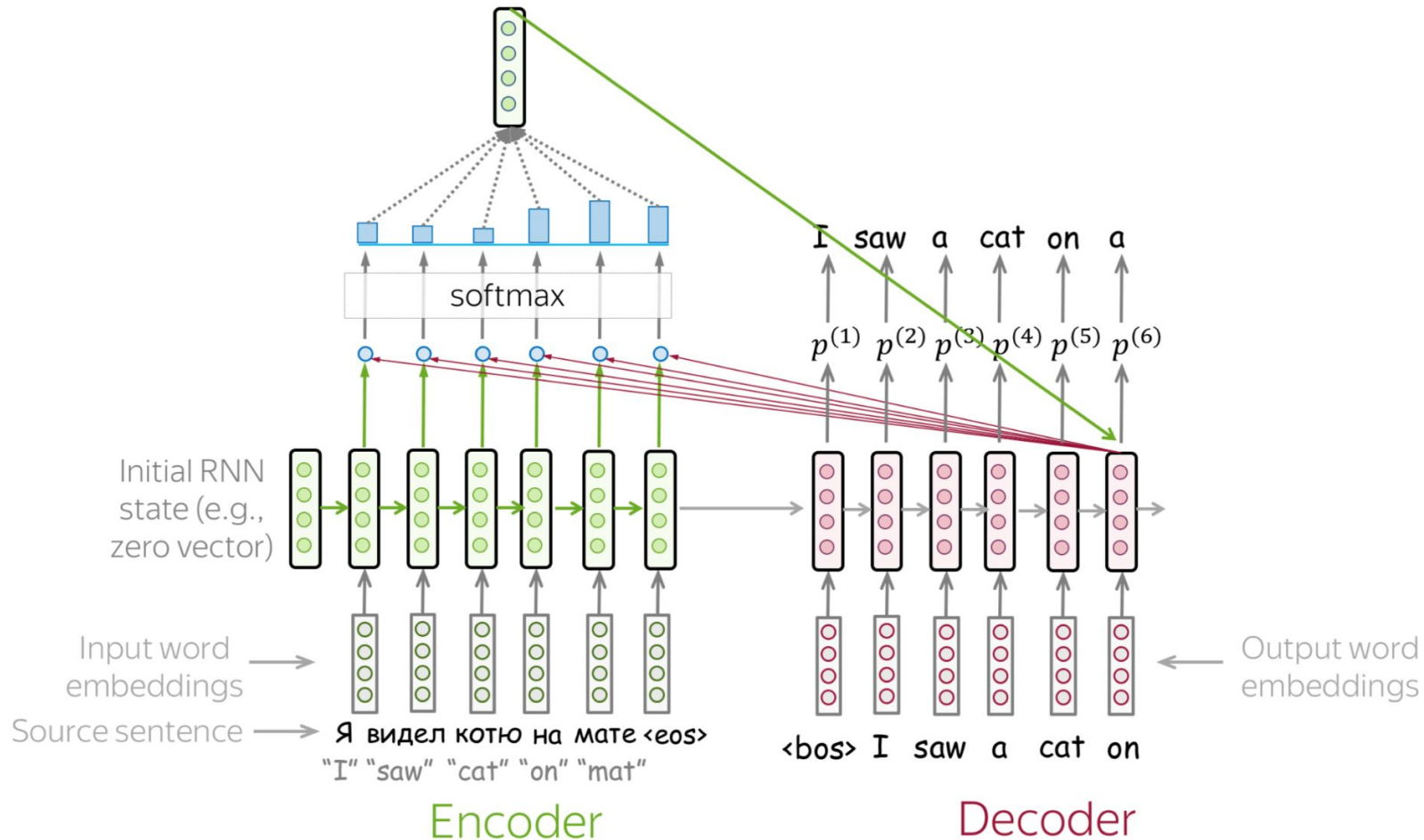
Attention: Steps



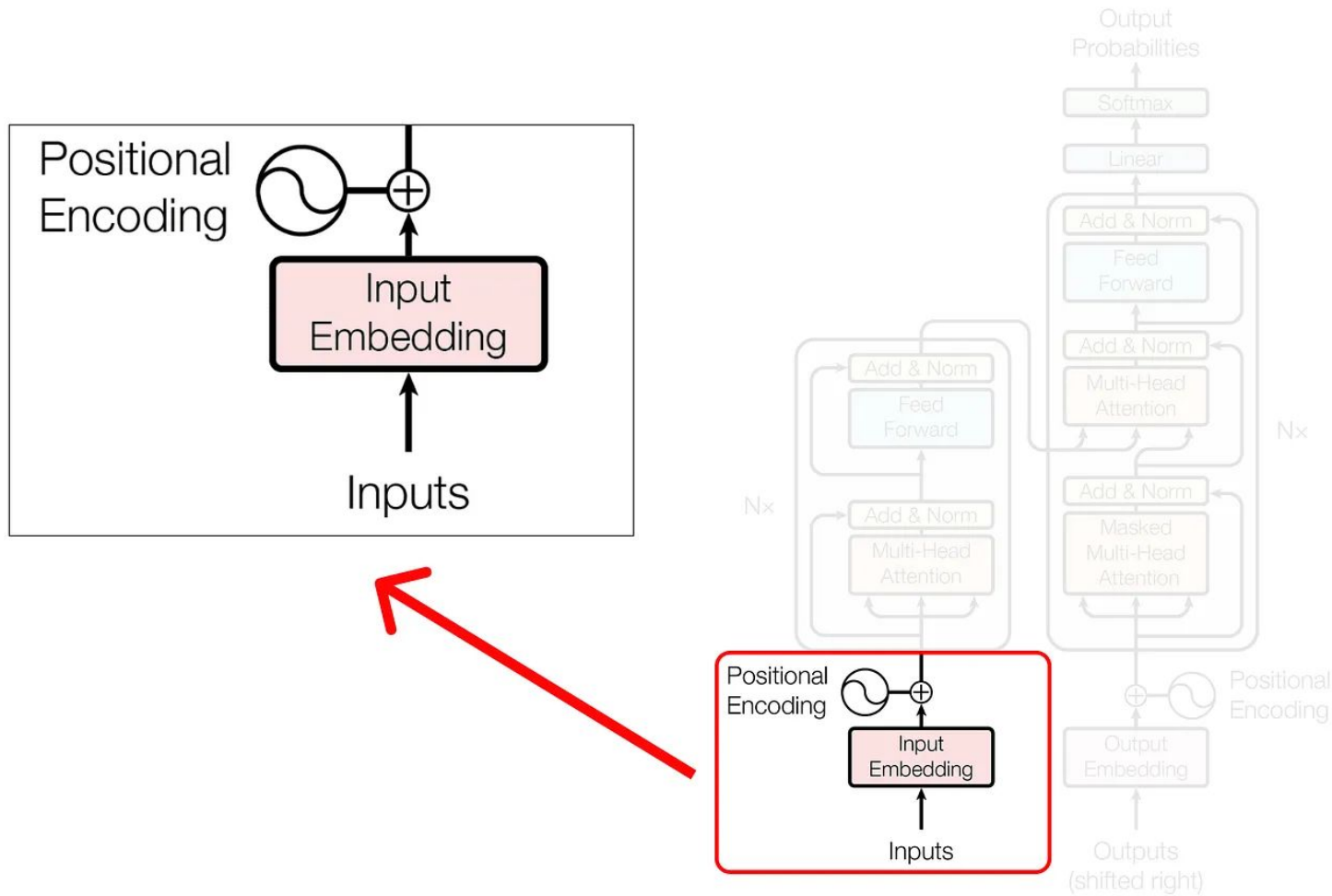
Attention: Steps



Attention: Steps



Inputs and Positional Encoding



Inputs and Positional Encoding (cnt.)

Dataset (Courpus)

1. I drink and I know things.
2. When you play the game of thrones, you win or you die.
3. The true enemy won't wait out the storm, He brings the storm.

$$vocab\ size = count(set(N))$$

Dataset (Courpus)

I drink and I know things.
When you play the game of thrones, you win or you die.
The true enemy won't wait out the storm, He brings the storm.

$$N = \begin{bmatrix} \text{I, drink, and, I, Know, things,} \\ \text{When, you, play, the, game, of, thrones, you, win, or, die,} \\ \text{The, true, enemy, won't, wait, out, the, storm, He, brings, the storm} \end{bmatrix}$$

$$vocab\ size = count(set(N))$$

$$\begin{aligned} &\rightarrow \text{set} (\text{I, drink, and, I, Know,} \\ &\quad \text{When, you, play, the, game, of, thrones, you, win, or, die} \\ &\quad \text{The, true, enemy, won't, wait, out, the, storm, He, brings, the storm}) \\ &\rightarrow \text{count} (\text{I, drink, and, Know, When, you, play, the, game, of, thrones,} \\ &\quad \text{win, or, die, true, enemy, won't, wait, out, storm, He, brings,} \\ &\quad \text{storm}) \\ &\rightarrow = 23 \end{aligned}$$

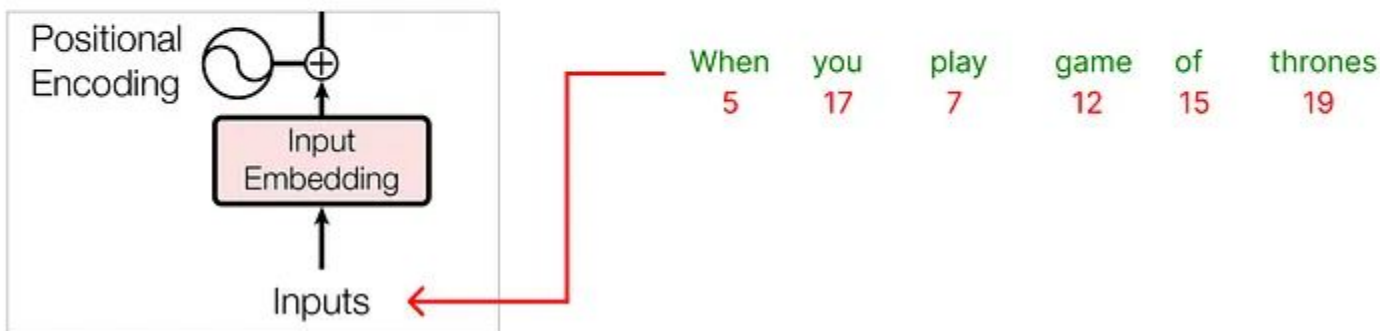
Inputs and Positional Encoding (cnt.)

1	2	3	4	5	6	7	8	9	10	11	12
I	drink	storm	Know	When	won't	play	out	true	storm	brings	game

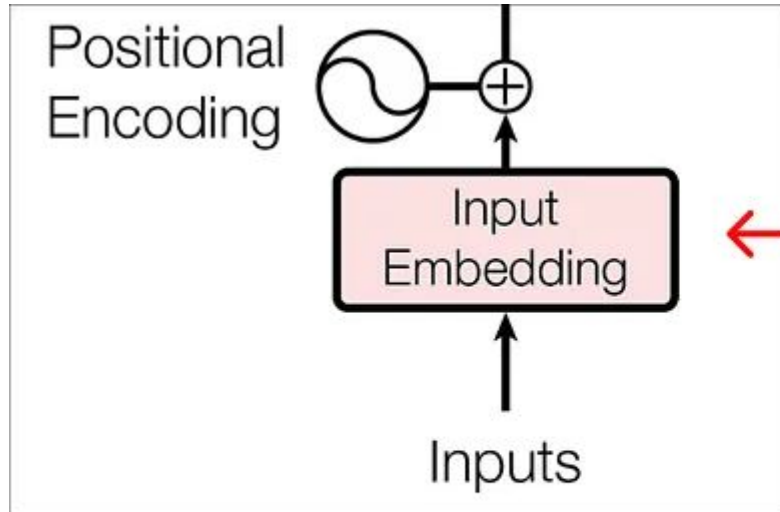
13	14	15	16	17	18	19	20	21	22	23
the	win	of	enemy	you	wait	thrones	and	or	die	He

1 2 3 4 5 6 7 8 9 10 11 12
I drink storm Know When won't play out true storm brings game

13 14 15 16 17 18 19 20 21 22 23
the win of enemy you wait thrones and or die He



Inputs and Positional Encoding (cnt.)



When	you	play	game	of	thrones
5	17	7	12	15	19
e1	e2	e3	e4	e5	e6
0.19	0.07	0.94	0.43	0.8	0.34
0.6	0.7	0.73	0.49	0.65	0.72
0.45	0.96	0.02	0.09	0.57	0.79
0.56	0.97	0.85	0.21	0.84	0.75
0.13	0.58	0.16	0.26	0.05	0.75

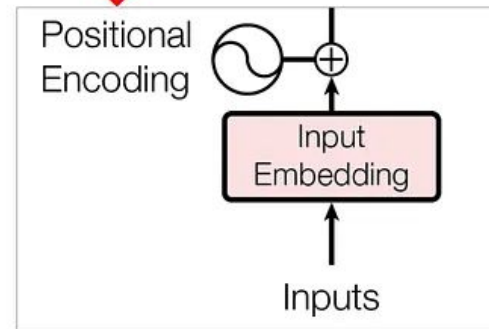
Inputs and Positional Encoding (cnt.)

$$PE(Odd) = \sin(pos/10000^{2i/dim})$$

$$PE(Even) = \cos(pos/10000^{2i/dim})$$

For Single word Calculation of positional Embedding

	When					
	5					
	e1					p1
i = 0	0.19	$PE(Even) = \cos(pos/10000^{2i/dim})$	$\cos(0/10000^{(2*0)/5})$			1
i = 1	0.6	$PE(Odd) = \sin(pos/10000^{2i/dim})$	$\sin(0/10000^{(2*1)/5})$			0
i = 2	0.45	$PE(Even) = \cos(pos/10000^{2i/dim})$	$\cos(0/10000^{(2*2)/5})$			1
i = 3	0.56	$PE(Odd) = \sin(pos/10000^{2i/dim})$	$\sin(0/10000^{(2*3)/5})$			0
i = 4	0.13	$PE(Even) = \cos(pos/10000^{2i/dim})$	$\cos(0/10000^{(2*4)/5})$			1
	dim (d) = 5					
	pos = 0					



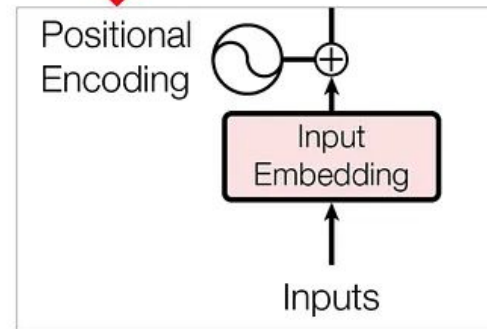
Inputs and Positional Encoding (cnt.)

$$PE(Odd) = \sin(pos/10000^{2i/dim})$$

$$PE(Even) = \cos(pos/10000^{2i/dim})$$

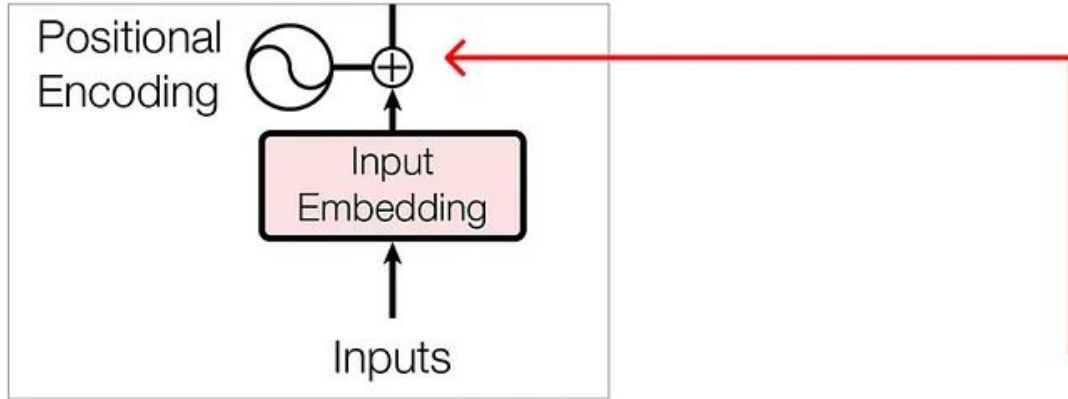
for all words in our first input, finding positional embedding

	When	you	play	game	of	thrones
	5	17	7	12	15	19
	p1	p2	p3	p4	p5	p6
	1	0.540302	-0.41615	-0.989992497	-0.65364362	0.283662185
	0	0.025116	0.050217	0.075285293	0.100306487	0.125264396
	1	1	0.999999	0.999998209	0.999996815	0.999995024
	0	1.58E-05	3.17E-05	4.75468E-05	6.33957E-05	7.92447E-05
	1	1	1	1	1	1
POS =	0	1	2	3	4	5
dim (d) =	5	5	5	5	5	5



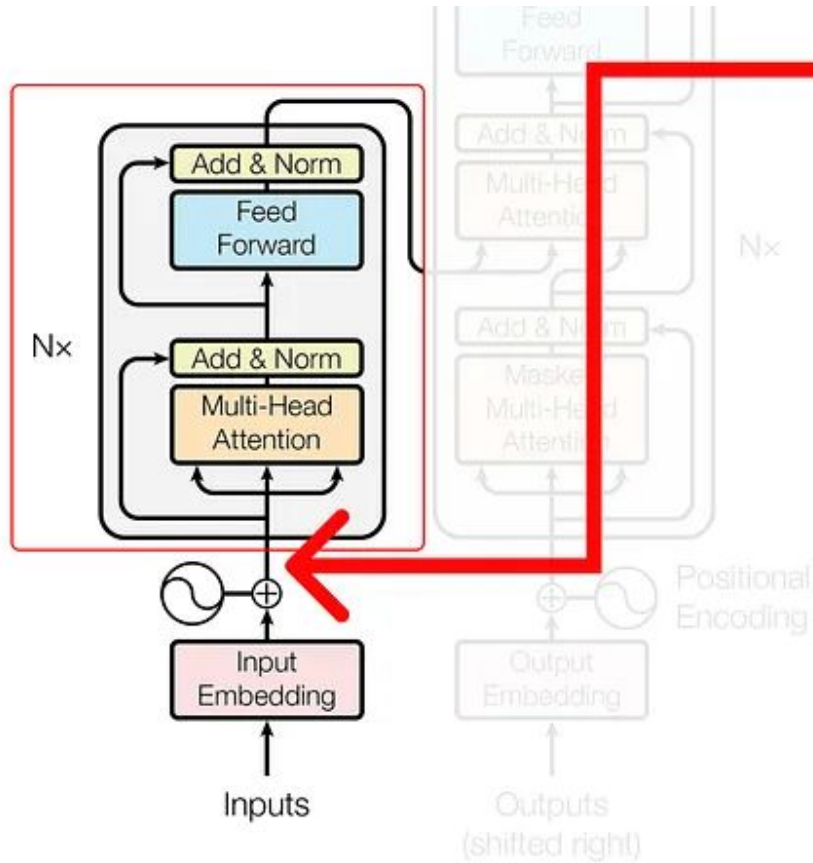
Inputs and Positional Encoding (cnt.)

Concatenate (Adding original Embedding and Positional Encoding)



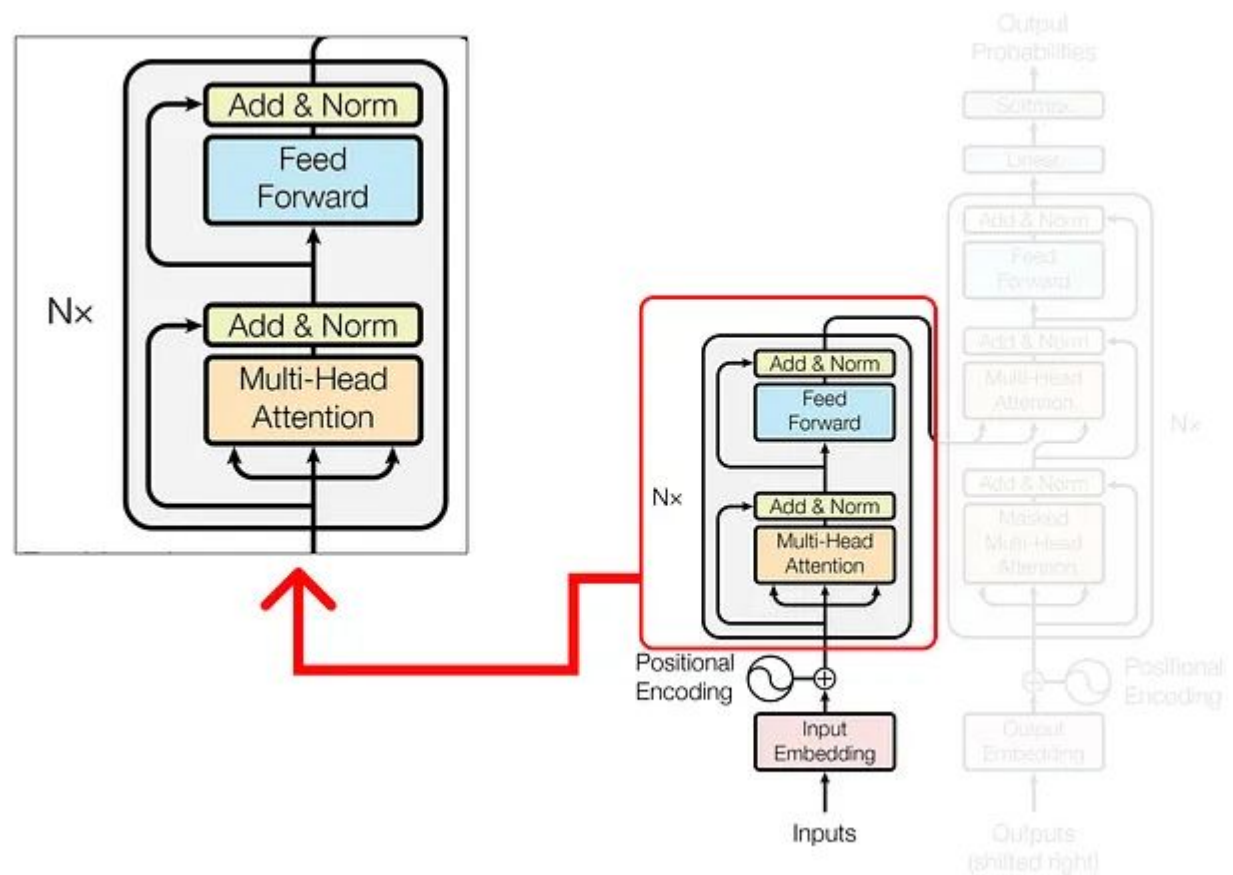
When	you	play	game	of	thrones
5	17	7	12	15	19
e1	e2	e3	e4	e5	e6
0.19	0.07	0.94	0.43	0.8	0.34
0.6	0.7	0.73	0.49	0.65	0.72
0.45	0.96	0.02	0.09	0.57	0.79
0.56	0.97	0.85	0.21	0.84	0.75
0.13	0.58	0.16	0.26	0.05	0.75
+	+	+	+	+	+
p1	p2	p3	p4	p5	p6
1	0.540302306	-0.416146837	-0.989992497	-0.653643621	0.283662185
0	0.025116223	0.050216599	0.075285293	0.100306487	0.125264396
1	0.999999801	0.999999204	0.999998209	0.999996815	0.999995024
0	1.58489E-05	3.16979E-05	4.75468E-05	6.33957E-05	7.92447E-05
1	1	1	1	1	1

Inputs and Positional Encoding (cnt.)

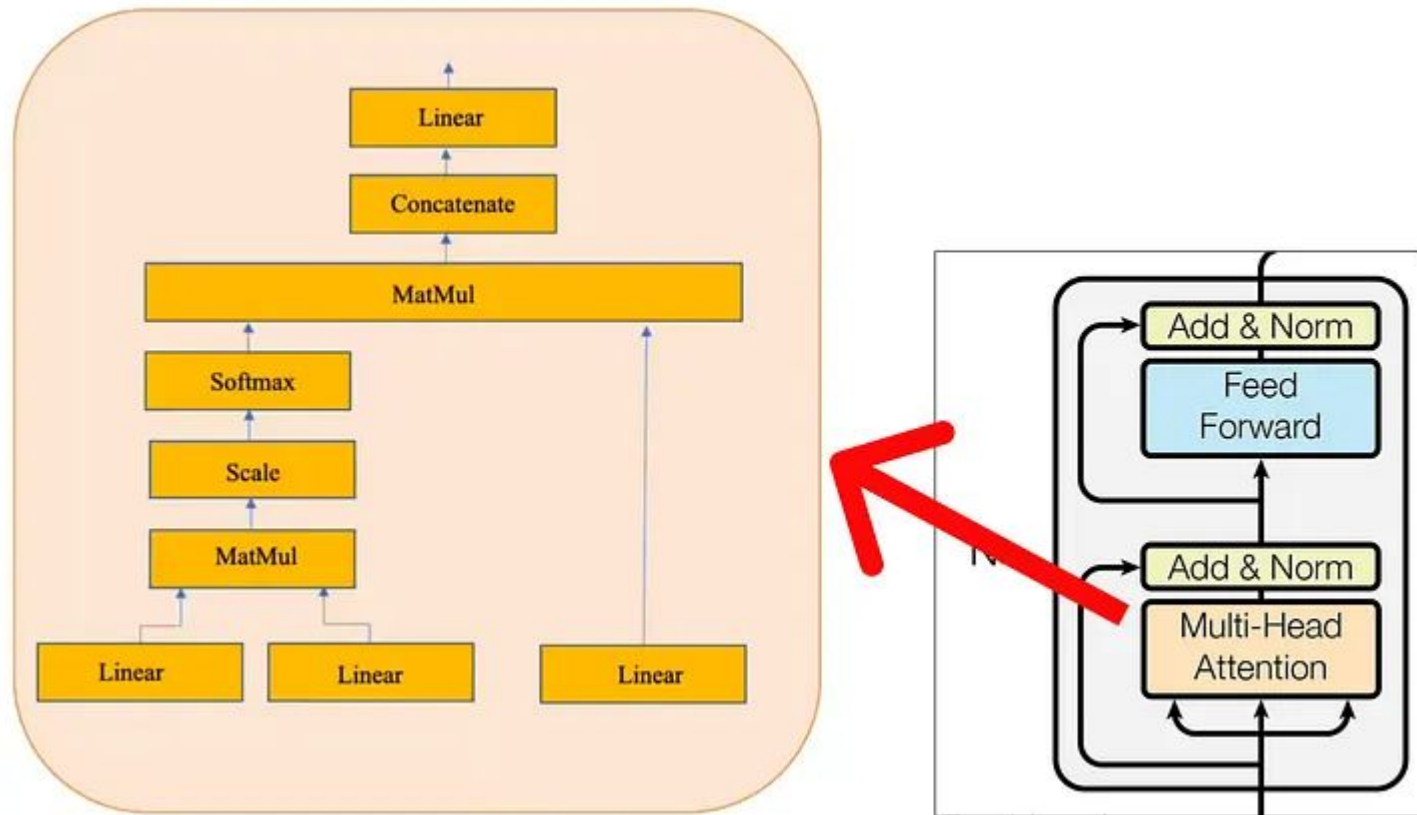


When	you	play	game	of	thrones
5	17	7	12	15	19
ep1	ep2	ep3	ep4	ep5	ep6
1.19	0.6103	0.5239	-0.56	0.1464	0.6237
0.6	0.7251	0.7802	0.5653	0.7503	0.8453
1.45	1.96	1.02	1.09	1.57	1.79
0.56	0.97	0.85	0.21	0.8401	0.7501
1.13	1.58	1.16	1.26	1.05	1.75

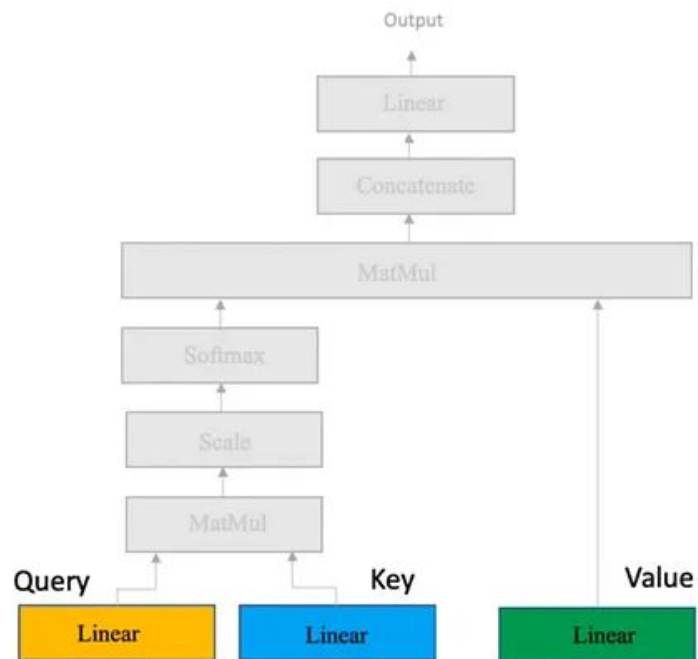
Encoder



Encoder



Encoder



	6 x 5		5 x 3		6 x 3
when					
you					
play					
game					
of					
thrones					
	Embedding + Positional		Linear weights		query

	6 x 5		5 x 3		6 x 3
when					
you					
play					
game					
of					
thrones					
	Embedding + Positional		Linear weights		key

	6 x 5		5 x 3		6 x 3
when					
you					
play					
game					
of					
thrones					
	Embedding + Positional		Linear weights		value

Encoder

	Embedding + Positional						Linear weights				Query		
when	1.19	0.6	1.5	0.56	1.13		0.552	0.26	0.208		2.8	2.3	2.44
you	0.61	0.73	2	0.97	1.58		0.885	0.21	0.262		3.3	3	3.33
play	0.52	0.78	1	0.85	1.16	X	0.178	0.89	0.297	=	2.7	2.1	2.54
game	-0.56	0.57	1.1	0.21	1.26		0.605	0.9	0.994		1.7	1.2	1.74
of	0.15	0.75	1.6	0.84	1.05		0.906	0.07	0.931		2.5	2.4	2.51
thrones	0.62	0.85	1.8	0.75	1.75						3.5	2.7	3.26

	Embedding + Positional						Linear weights				Key		
when	1.19	0.6	1.5	0.56	1.13		0.996	0.22	0.237		2.9	1.6	1.9
you	0.61	0.73	2	0.97	1.58		0.667	0.87	0.785		3	2	2.45
play	0.52	0.78	1	0.85	1.16	X	0.587	0.1	0.15	=	2.3	1.6	2.06
game	-0.56	0.57	1.1	0.21	1.26		0.565	0.17	0.793		0.7	1.2	1.18
of	0.15	0.75	1.6	0.84	1.05		0.122	0.52	0.427		2.2	1.5	1.97
thrones	0.62	0.85	1.8	0.75	1.75						2.9	2.1	2.42

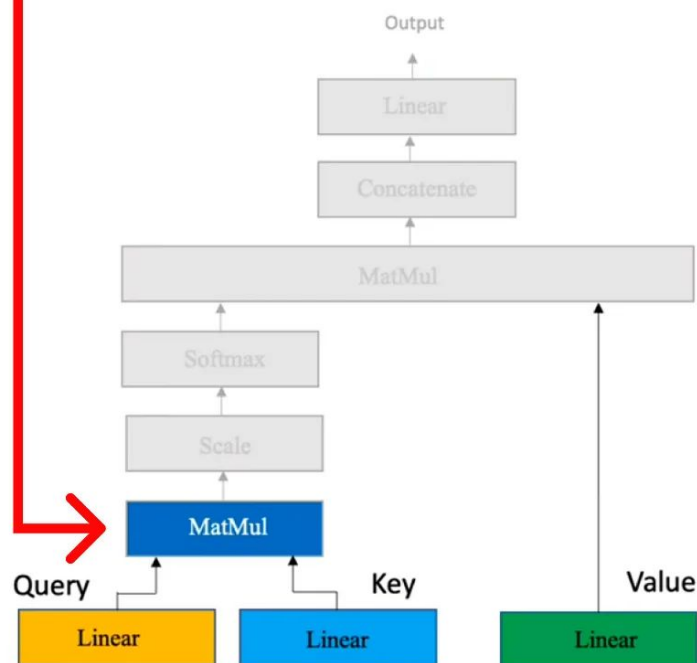
	Embedding + Positional						Linear weights				Value		
when	1.19	0.6	1.5	0.56	1.13		0.896	0.58	0.019		3	2.8	1.98
you	0.61	0.73	2	0.97	1.58		0.045	0.43	0.594		3.3	3.3	2.7
play	0.52	0.78	1	0.85	1.16	X	0.717	0.52	0.495	=	2.3	2.4	1.97
game	-0.56	0.57	1.1	0.21	1.26		0.658	0.27	0.288		1	1.6	1.73
of	0.15	0.75	1.6	0.84	1.05		0.443	0.87	0.64		2.3	2.4	2.14
thrones	0.62	0.85	1.8	0.75	1.75						3.2	3.4	2.74

Encoder

$$query * transpose(key)$$

Query				Transpose(Key)					
2.8	2.31	2.44							
3.35	3.04	3.33		2.89	2.98	2.26	0.731047	2.170039	2.872435
2.73	2.05	2.54	X	1.62	1.95	1.64	1.166483	1.531395	2.091971
1.65	1.22	1.74		1.9	2.45	2.06	1.179864	1.973794	2.422168
2.48	2.42	2.51							
3.45	2.73	3.26							

16.46063	18.84472	15.16177	7.622069	14.4337	18.78968
20.91132	24.08375	19.43355	9.924581	18.49635	24.04522
16.02201	18.36819	14.7777	7.386308	14.07696	18.28246
10.05137	11.57518	9.327719	4.683798	8.888656	11.51414
15.85299	18.28119	14.76566	7.59949	14.04828	18.27532
20.56783	23.60403	19.00353	9.550073	18.09826	23.51283



Encoder

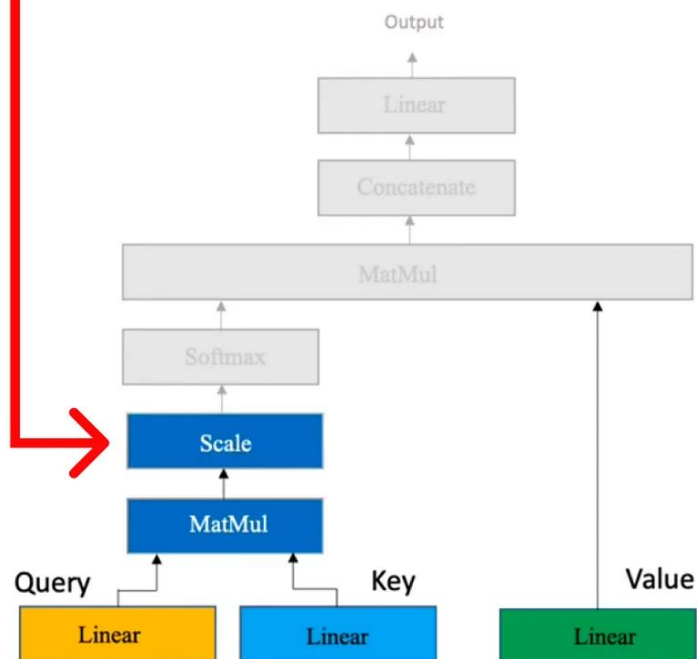
16.46063	18.84472	15.16177	7.622069	14.4337	18.78968
20.91132	24.08375	19.43355	9.924581	18.49635	24.04522
16.02201	18.36819	14.7777	7.386308	14.07696	18.28246
10.05137	11.57518	9.327719	4.683798	8.888656	11.51414
15.85299	18.28119	14.76566	7.59949	14.04828	18.27532
20.56783	23.60403	19.00353	9.550073	18.09826	23.51283

$$\sqrt{d_k}$$

where d is 5 (dimension of embedding vector)

=

3.3	3.77	3.03	1.52	2.89	3.76
4.2	4.82	3.89	1.98	3.7	4.81
3.2	3.67	2.96	1.48	2.82	3.66
2	2.32	1.87	0.94	1.78	2.3
3.2	3.66	2.95	1.52	2.81	3.66
4.1	4.72	3.8	1.91	3.62	4.7



Encoder

3.3	3.77	3.03	1.52	2.89	3.76
4.2	4.82	3.89	1.98	3.7	4.81
3.2	3.67	2.96	1.48	2.82	3.66
2	2.32	1.87	0.94	1.78	2.3
3.2	3.66	2.95	1.52	2.81	3.66
4.1	4.72	3.8	1.91	3.62	4.7

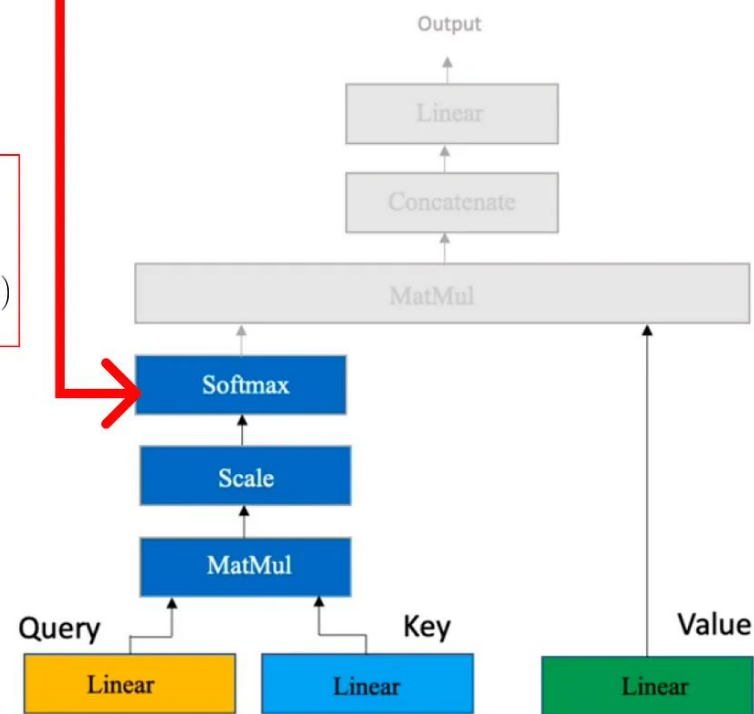
$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_i)}$$

$$\exp(3.3)$$

$$\exp(3.3) + \exp(3.77) + \exp(3.03) + \exp(1.52) + \exp(2.89) + \exp(3.76)$$

=

0.17	0.33	0.235	0.07	0.27	0.88
0.16	0.36	0.22	0.04	0.24	0.95
0.17	0.33	0.237	0.07	0.28	0.87
0.18	0.28	0.238	0.12	0.29	0.63
0.17	0.33	0.236	0.07	0.28	0.87
0.16	0.36	0.222	0.04	0.24	0.95



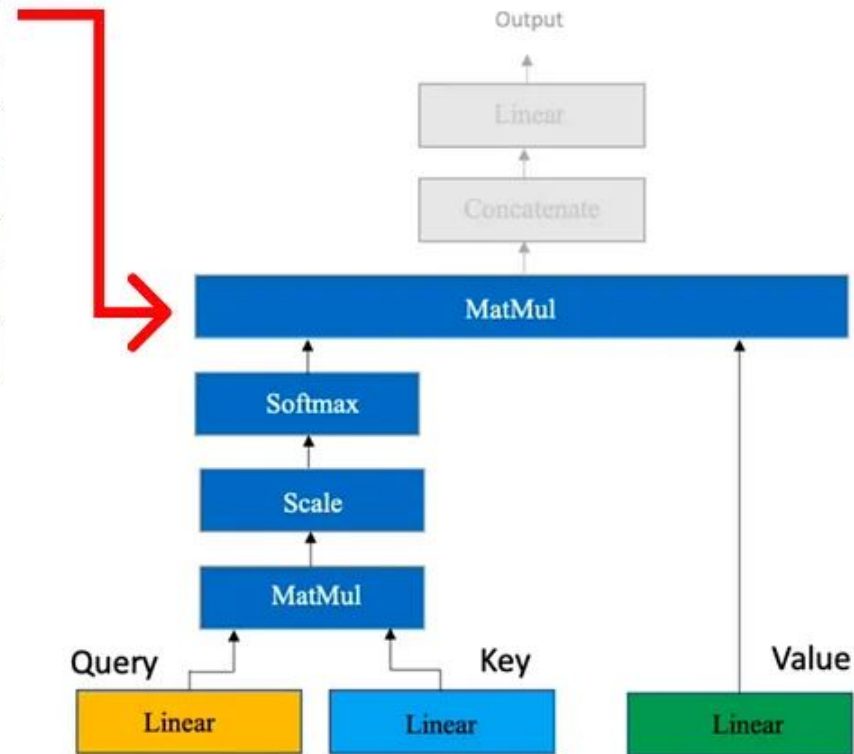
Encoder

$$\text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)$$

resultant matrix							Value		
0.17	0.33	0.23	0.07	0.27	0.88		3	3.4	2.06
0.16	0.36	0.22	0.04	0.24	0.95		3.97	3.7	2.84
0.17	0.33	0.24	0.07	0.28	0.87	X	2.64	2.6	2.24
0.18	0.28	0.24	0.12	0.29	0.63		2.46	1.3	1.49
0.17	0.33	0.24	0.07	0.28	0.87		3.02	2.7	2.19
0.16	0.36	0.22	0.04	0.24	0.95		3.93	3.6	2.78

=

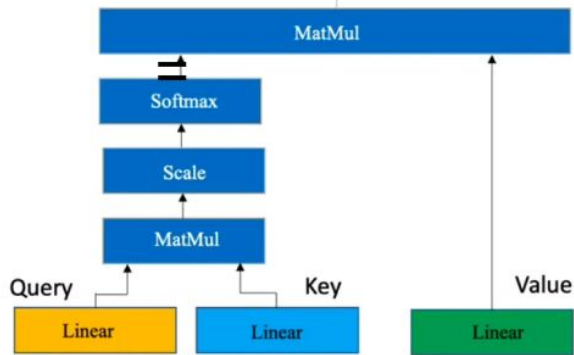
6.89	6.4	4.96
7.07	6.57	5.08
6.86	6.37	4.94
5.89	5.43	4.24
6.84	6.34	4.92
7.08	6.59	5.09



Encoder

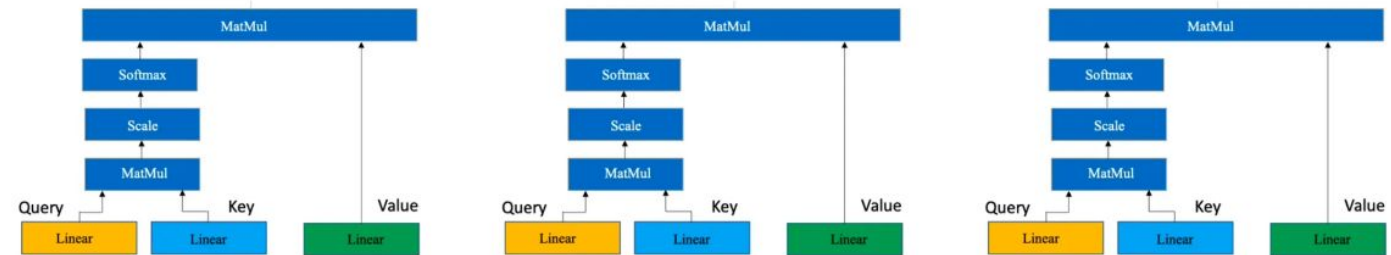
Our case (no concatenation single head)

6.89	6.4	4.96
7.07	6.57	5.08
6.86	6.37	4.94
5.89	5.43	4.24
6.84	6.34	4.92
7.08	6.59	5.09



Real world case (concatenation multi head)

6.89	6.4	4.96
7.07	6.57	5.08
6.86	6.37	4.94
5.89	5.43	4.24
6.84	6.34	4.92
7.08	6.59	5.09

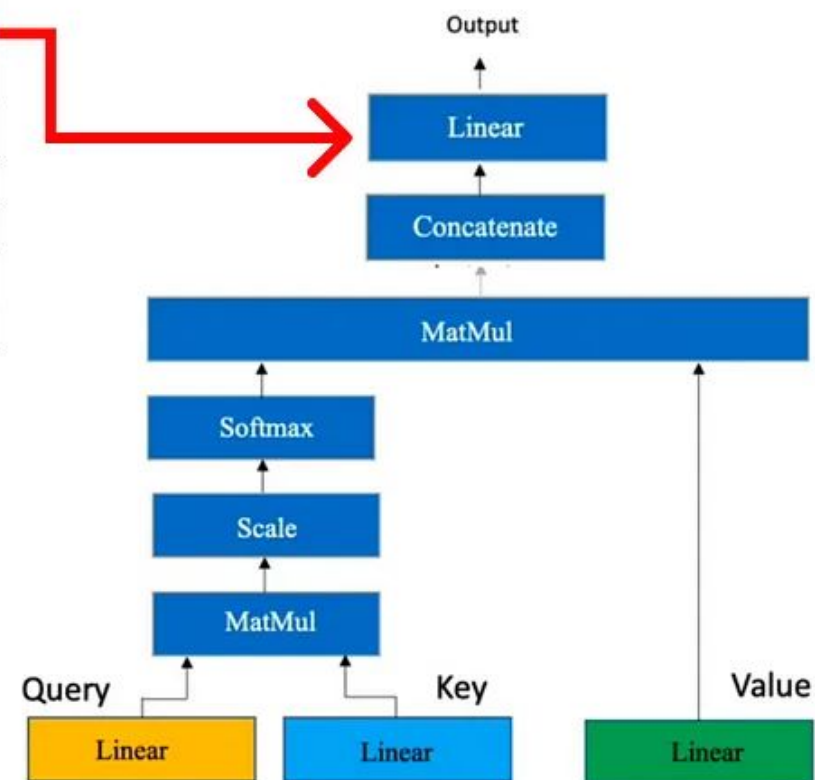


Encoder

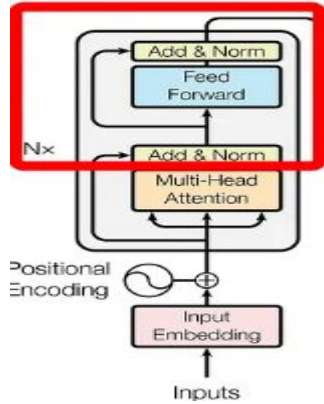
$\text{softmax}(Q.K^T/\sqrt{d}) * (V)$				random Linear weights			
6.89	6.399411	4.95966					
7.07	6.574745	5.082623		0.836	0.112	0.522	0.619044
6.86	6.369584	4.93844	X	0.809	0.408	0.019	0.120182
5.89	5.431721	4.235453		0.15	0.553	0.324	0.198002
6.84	6.342081	4.920775					
7.08	6.586521	5.090932					

=

11.68632	6.129	5.333	6.019
11.99128	6.288	5.467	6.172
11.63378	6.102	5.31	5.993
9.952935	5.221	4.554	5.136
11.58904	6.078	5.291	5.971
12.01129	6.299	5.476	6.182



Encoder Add&Norm – Feed Forward



* ADD&NORM:

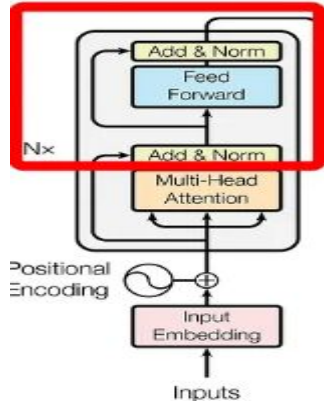
- Cùng nhớ lại kết quả có được sau khi ra khỏi giai đoạn self-attention:

11.68632	6.129	5.333	6.019	3.171
11.99128	6.288	5.467	6.172	3.254
11.63378	6.102	5.31	5.993	3.157
9.952935	5.221	4.554	5.136	2.702
11.58904	6.078	5.291	5.971	3.145
12.01129	6.229	5.476	6.182	3.26

- Tiếp theo nhớ lại Input Matrix trước khi đi vào giai đoạn Self-attention là:

1.19	0.6	1.5	0.56	1.13
0.61	0.73	2	0.97	1.58
0.52	0.78	1	0.85	1.16
-0.56	0.57	1.1	0.21	1.26
0.15	0.75	1.6	0.84	1.05
0.62	0.85	1.8	0.75	1.75

Encoder Add&Norm – Feed Forward



- Để thực hiện minh họa, ta sẽ chỉ xét chi tiết lớp add&norm sau khối Multihead Self-attention, còn lớp add&norm sau khối Feed-Forward sẽ hoàn toàn có cách tính tương tự. Ta có tổng quan các bước thực hiện của lớp Add&Norm sau khi kết thúc Self-Attention như sau:

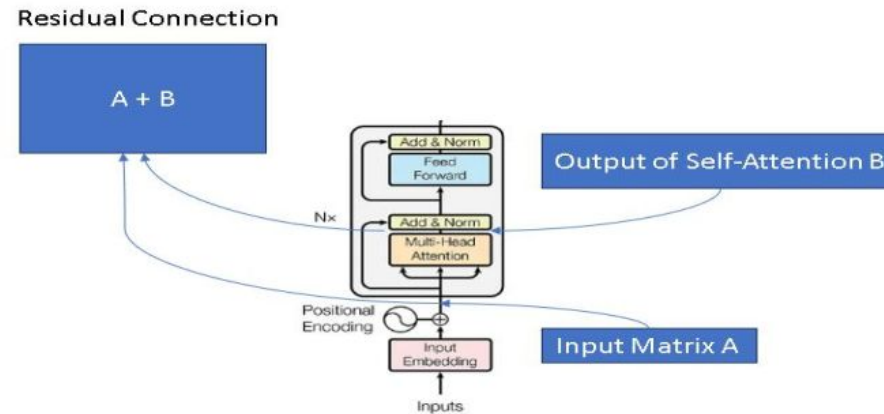
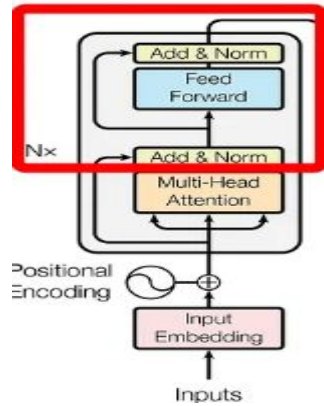
Layer Norm

11.68632	6.129	5.333	6.019	3.171
11.99128	6.288	5.467	6.172	3.254
11.63378	6.102	5.31	5.993	3.157
9.952935	5.221	4.554	5.136	2.702
11.58904	6.078	5.291	5.971	3.145
12.01129	6.229	5.476	6.182	3.26

+

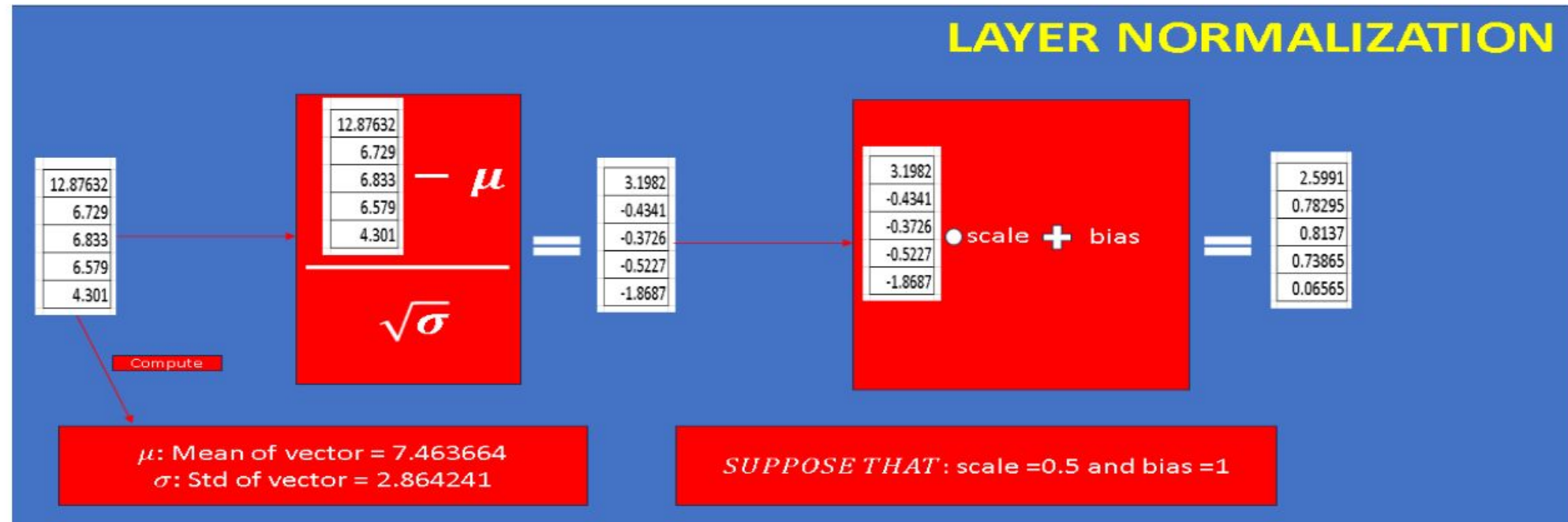
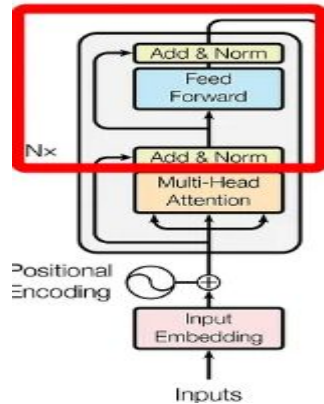
1.19	0.6	1.5	0.56	1.13
0.61	0.73	2	0.97	1.58
0.52	0.78	1	0.85	1.16
-0.56	0.57	1.1	0.21	1.26
0.15	0.75	1.6	0.84	1.05
0.62	0.85	1.8	0.75	1.75

Encoder Add&Norm – Feed Forward

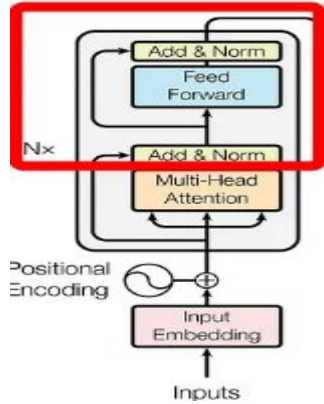


11.68632	6.129	5.333	6.019	3.171	+	1.19	0.6	1.5	0.56	1.13	=	12.87632	6.729	6.833	6.579	4.301
11.99128	6.288	5.467	6.172	3.254		0.61	0.73	2	0.97	1.58		12.60128	7.018	7.467	7.142	4.834
11.63378	6.102	5.31	5.993	3.157		0.52	0.78	1	0.85	1.16		12.15378	6.882	6.31	6.843	4.317
9.952935	5.221	4.554	5.136	2.702		-0.56	0.57	1.1	0.21	1.26		9.392935	5.791	5.654	5.346	3.962
11.58904	6.078	5.291	5.971	3.145		0.15	0.75	1.6	0.84	1.05		11.73904	6.828	6.891	6.811	4.195
12.01129	6.229	5.476	6.182	3.26		0.62	0.85	1.8	0.75	1.75		12.63129	7.079	7.276	6.932	5.01

Encoder Add&Norm – Feed Forward



Encoder Add&Norm – Feed Forward

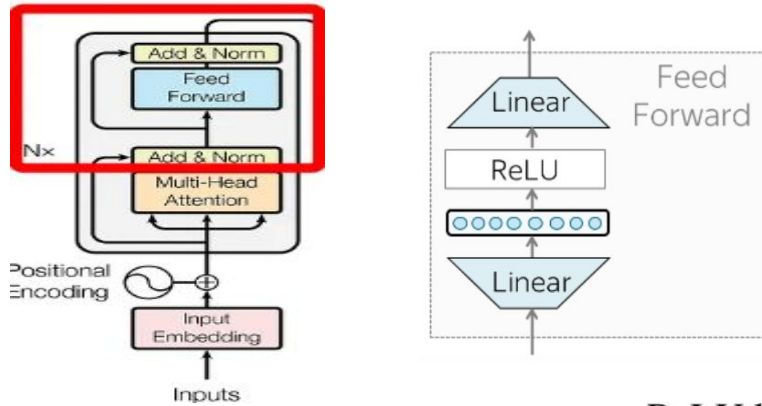


12.87632	6.729	6.833	6.579	4.301
12.60128	7.018	7.467	7.142	4.834
12.15378	6.882	6.31	6.843	4.317
9.392935	5.791	5.654	5.346	3.962
11.73904	6.828	6.891	6.811	4.195
12.63129	7.079	7.276	6.932	5.01

Norm

2.5991	0.78295	0.8137	0.73865	0.06565
2.4938	0.7522	0.8922	0.7909	0.0709
2.5045	0.87	0.6927	0.858	0.0748
2.2526	0.9113	0.8603	0.7456	0.2302
2.4207	0.8515	0.8716	0.846	0.0101
3.0307	0.779	0.8406	0.733	0.132

Encoder Add&Norm – Feed Forward



- ReLU là gì?

+ Tên đầy đủ là Rectified Linear Unit, được tính theo công thức:

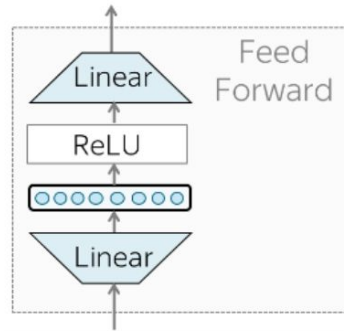
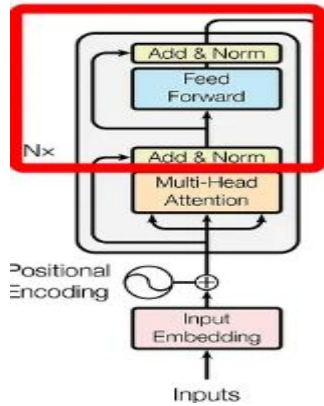
$$f(x) = \max(0, x)$$

+ Hàm ReLU đơn giản là loại bỏ các giá trị nhỏ hơn 0.

- Vậy nếu ta gọi kết quả thu được từ bước Add&Norm trước đó là $X_{\text{selfatn_addnorm}}$ thì công thức tính tổng qua cho output của khối Feed forward sẽ là:

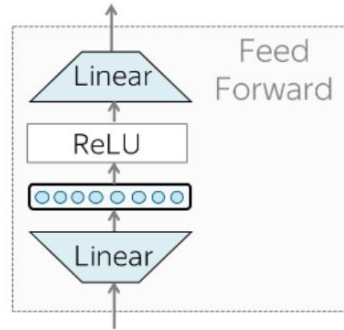
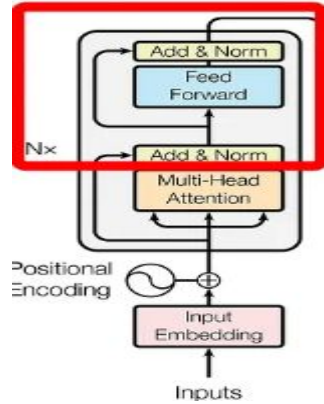
$$\text{Output} = \max(0, X_{\text{selfatn_addnorm}} W_1 + b_1) W_2 + b_2$$

Encoder Add&Norm – Feed Forward



	0.514	0.128	0.422	0.109	-0.106		
	-0.602	0.662	0.354	0.596	0.222		
W1 =	-0.828	0.456	0.226	0.207	0.317	b1 = 0	
	0.133	0.301	0.704	0.485	-0.423		
	-0.547	0.538	0.111	0.311	-0.281		
	0.403	0.115	0.465	0.203	0.466		
	0.102	0.206	0.318	0.215	0.382		
W2 =	0.289	0.721	0.627	0.342	0.517	b2 = 0	
	0.364	0.438	0.105	0.344	0.222		
	0.821	0.229	0.106	0.201	0.212		

Encoder Add&Norm – Feed Forward



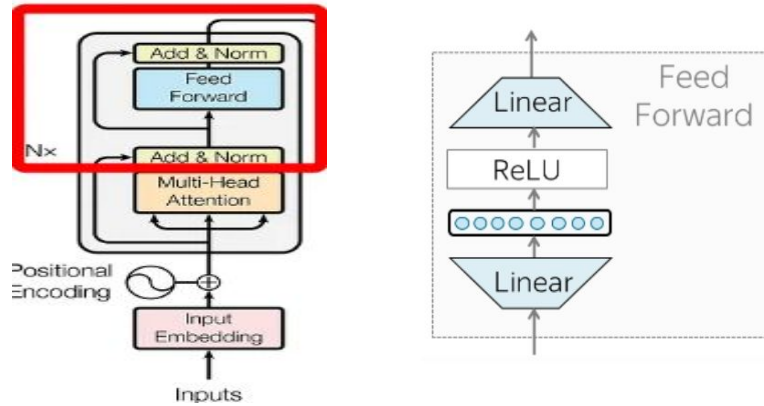
$$X_{selfatn_addnorm} W_1 =$$

0.2531878	1.47969825	2.08517745	1.2970384	-0.1746434
0.1566546	1.5002111	2.0849631	1.3104572	-0.1690006
0.2632158	1.5108876	2.133764	1.3742922	-0.2367039
-0.1298492	1.6321834	2.0180798	1.3999585	-0.1438269
0.1169453	1.531072	2.1166531	1.3652226	-0.1519601
0.41809	1.5785902	2.275381	1.3651915	-0.228997

$$\max(0, X_{selfatn_addnorm} W_1) =$$

0.2531878	1.47969825	2.08517745	1.2970384	0
0.1566546	1.5002111	2.0849631	1.3104572	0
0.2632158	1.5108876	2.133764	1.3742922	0
0	1.6321834	2.0180798	1.3999585	0
0.1169453	1.531072	2.1166531	1.3652226	0
0.41809	1.5785902	2.275381	1.3651915	0

Encoder Add&Norm – Feed Forward

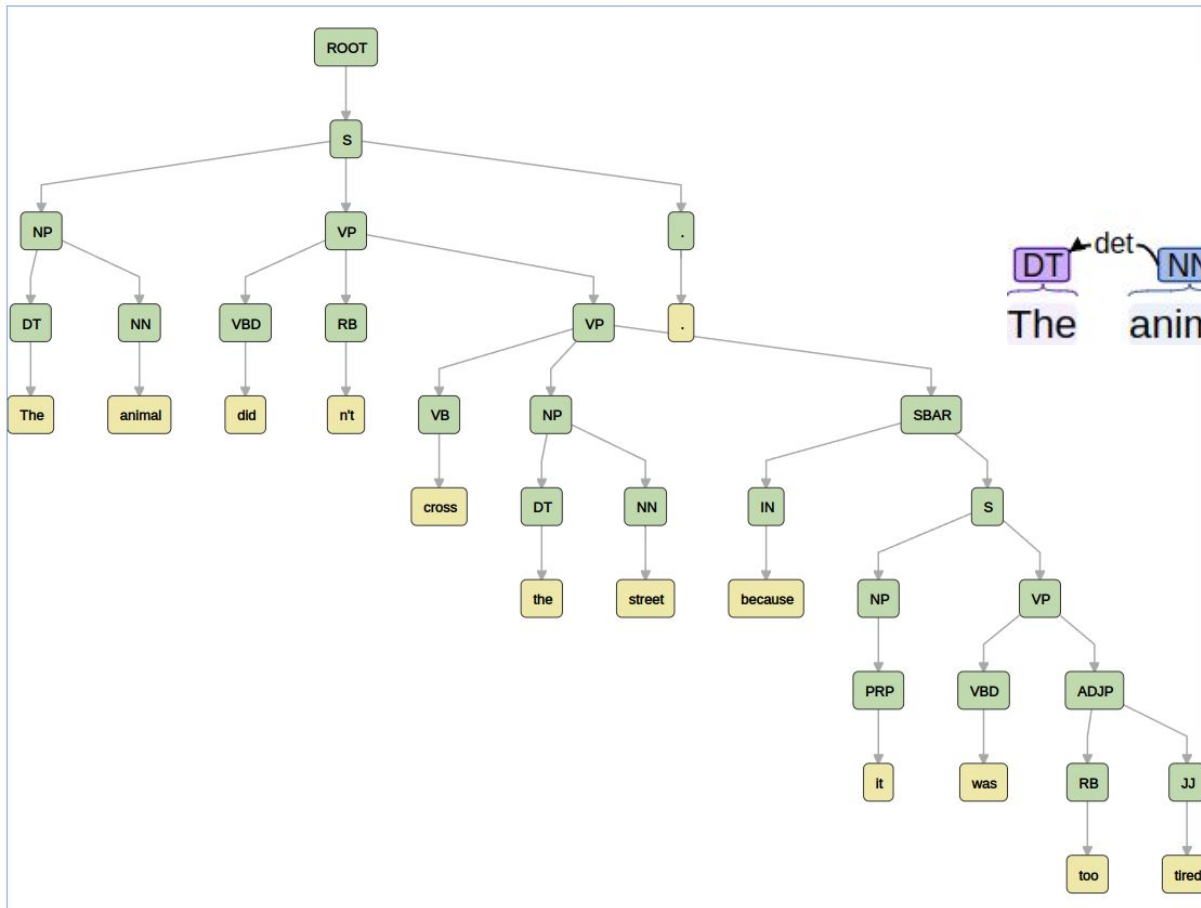


$$\text{Output} = \max(0, X_{\text{selfatn_addnorm}} W_1) W_2$$

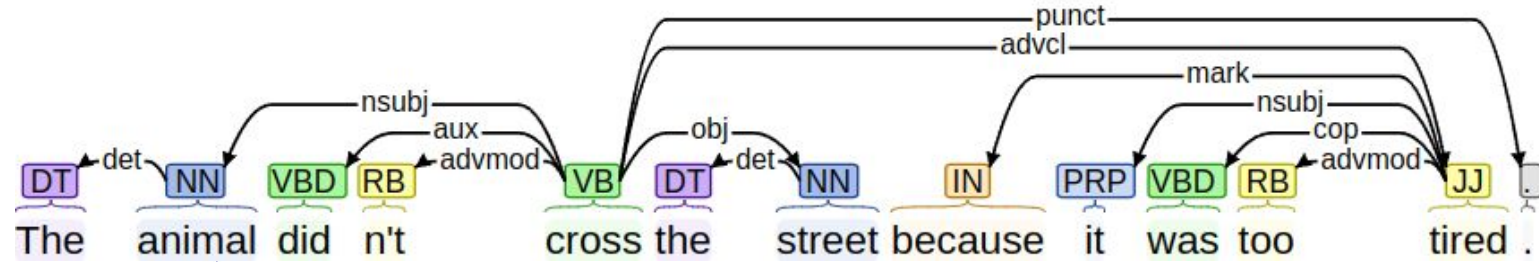
	1.328	2.405	2.032	1.529	2.049
	1.296	2.404	1.995	1.518	2.015
Output =	1.377	2.482	2.085	1.581	2.108
	1.259	2.404	1.931	1.523	1.978
	1.312	2.453	2.012	1.546	2.037
	1.484	2.612	2.266	1.672	2.277

From syntactic to dependency

Constituency Parse:

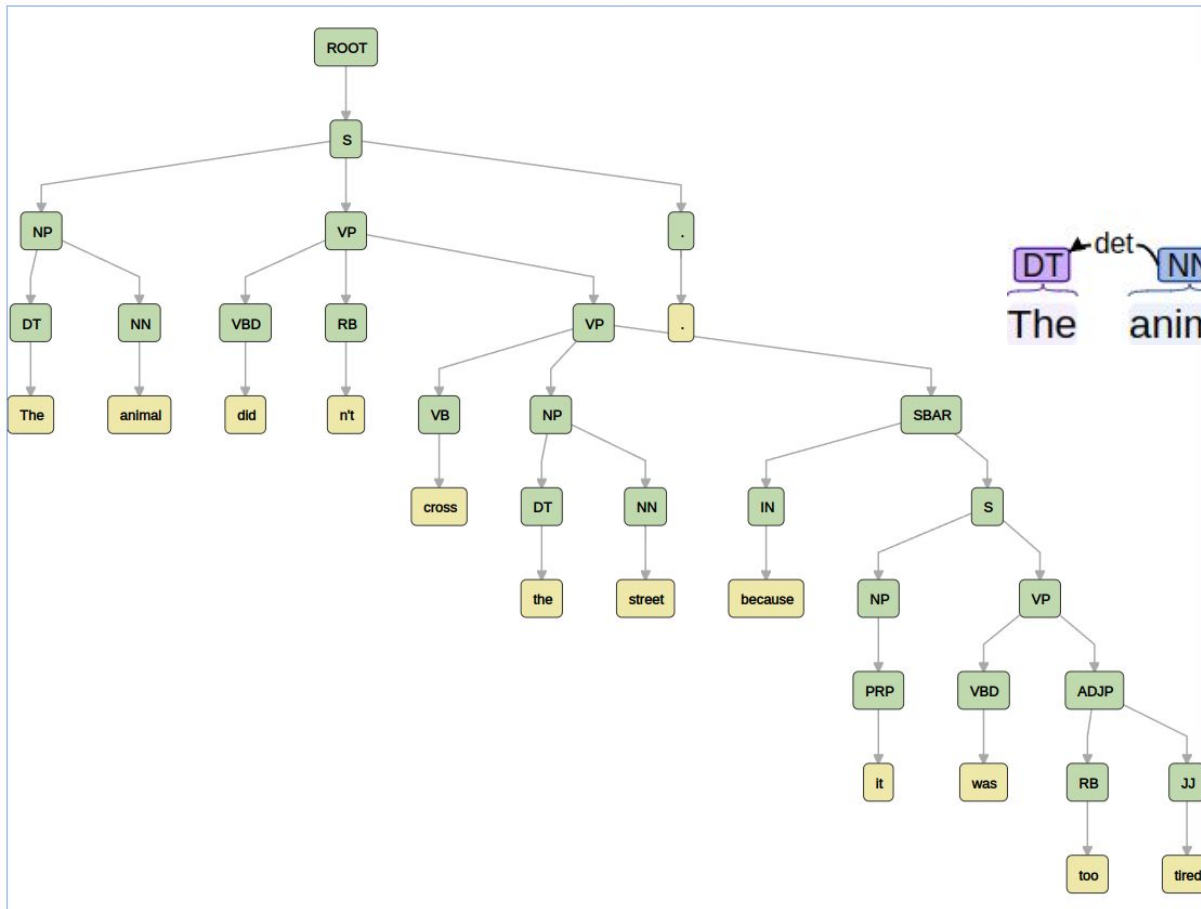


Dependency Parse:

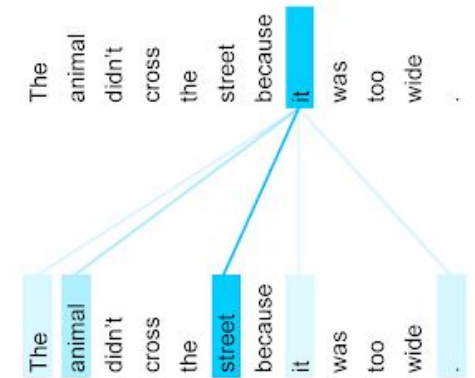
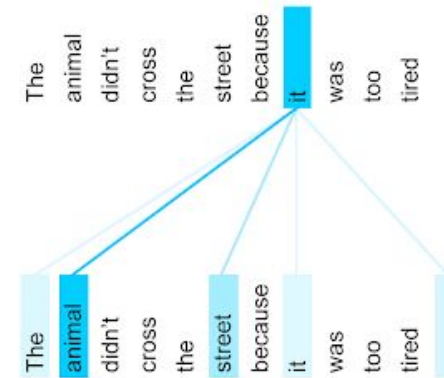
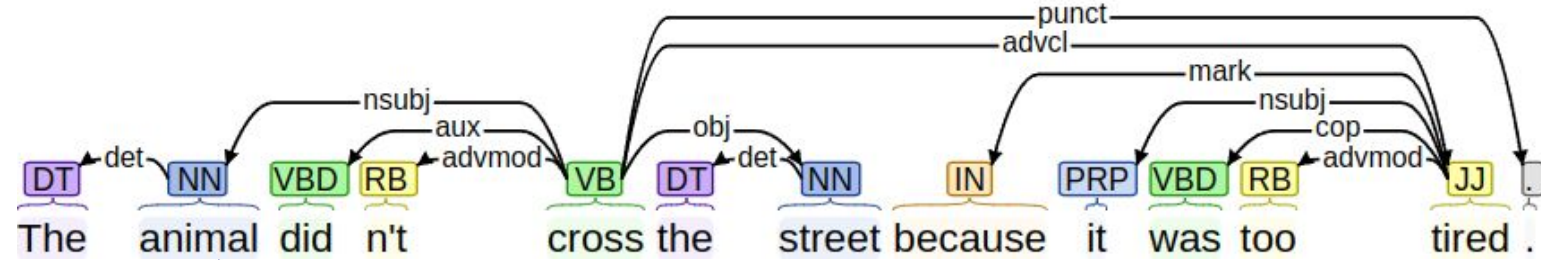


From dependency to SELF-ATTENTION

Constituency Parse:



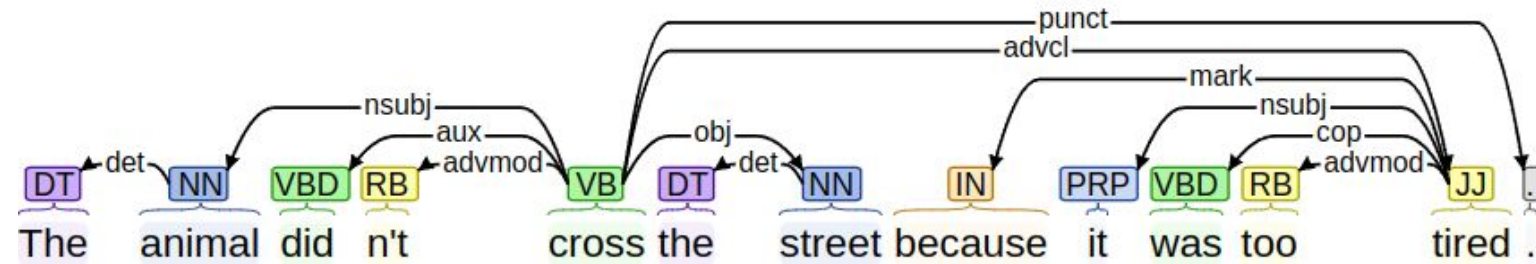
Dependency Parse:



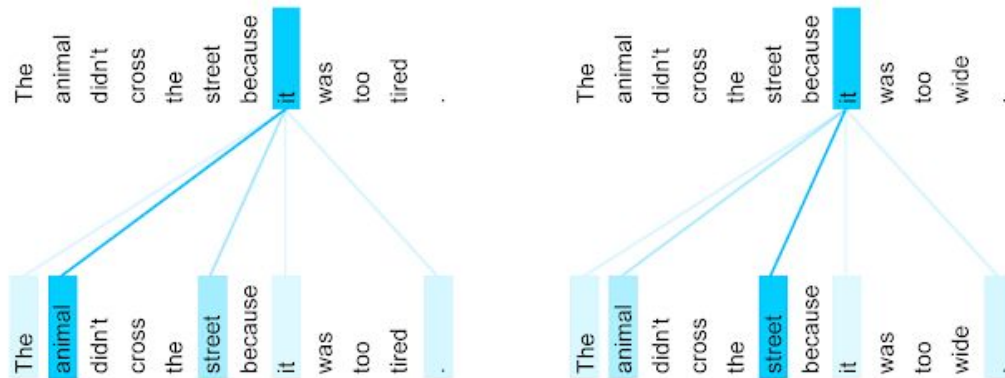
SELF-ATTENTION

From SELF-ATTENTION to FFN

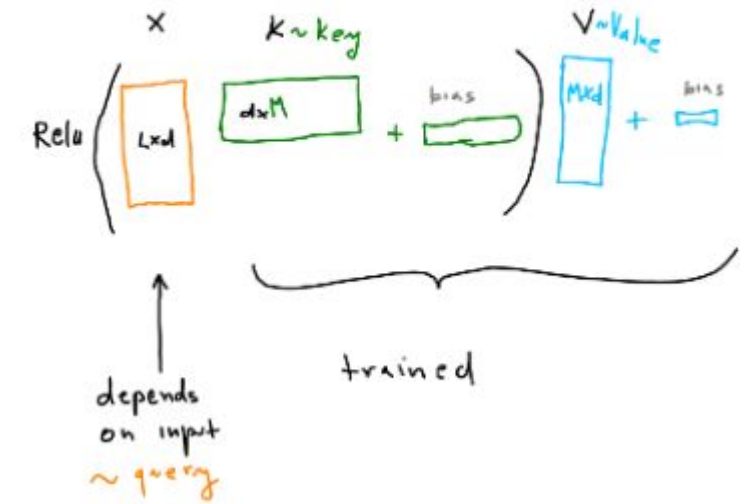
Dependency Parse:



SELF-ATTENTION



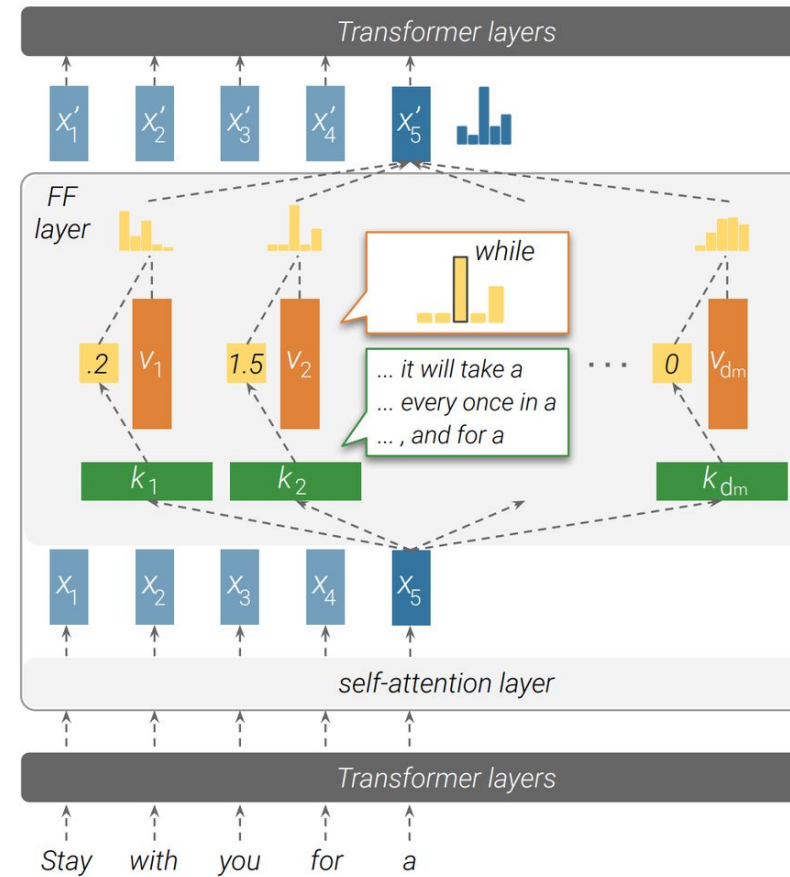
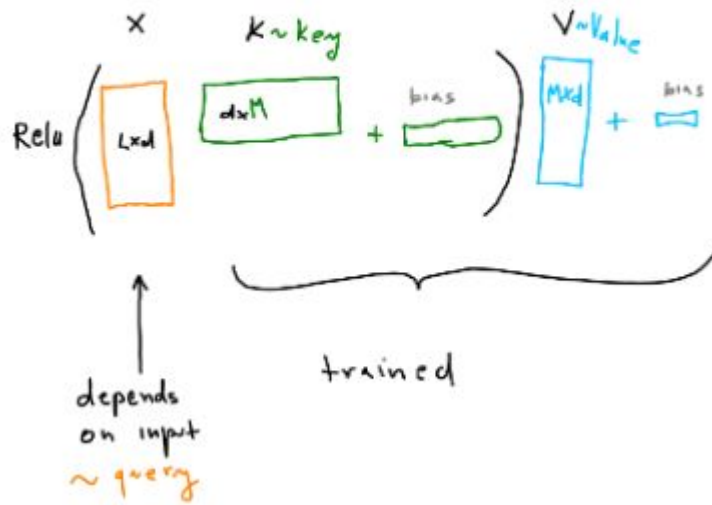
Transformer Feed-Forward



$$\text{FF}(\mathbf{x}) = f(\mathbf{x} \cdot \mathbf{K}^T) \cdot \mathbf{V}$$

From SELF-ATTENTION to FFN

Transformer Feed-Forward



From SELF-ATTENTION to FFN

Key	Pattern	Example trigger prefixes
k_{449}^1	Ends with “substitutes” (shallow)	<i>At the meeting, Elton said that “for artistic reasons there could be no substitutes In German service, they were used as substitutes Two weeks later, he came off the substitutes</i>
k_{2546}^6	Military, ends with “base”/“bases” (shallow + semantic)	<i>On 1 April the SRSG authorised the SADF to leave their bases Aircraft from all four carriers attacked the Australian base Bombers flying missions to Rabaul and other Japanese bases</i>
k_{2997}^{10}	a “part of” relation (semantic)	<i>In June 2012 she was named as one of the team that competed He was also a part of the Indian delegation Toy Story is also among the top ten in the BFI list of the 50 films you should</i>
k_{2989}^{13}	Ends with a time range (semantic)	<i>Worldwide, most tornadoes occur in the late afternoon, between 3 pm and 7 Weekend tolls are in effect from 7:00 pm Friday until The building is open to the public seven days a week, from 11:00 am to</i>
k_{1935}^{16}	TV shows (semantic)	<i>Time shifting viewing added 57 percent to the episode’s The first season set that the episode was included in was as part of the From the original NBC daytime version , archived</i>

Table 1: Examples of human-identified patterns that trigger different memory keys.

From SELF-ATTENTION to FFN

Key	Pattern	Example trigger prefixes
k_{449}^1	Ends with “substitutes” (shallow)	<i>At the meeting, Elton said that “for artistic reasons there could be no substitutes In German service, they were used as substitutes Two weeks later, he came off the substitutes</i>
k_{2546}^6	Military, ends with “base”/“bases” (shallow + semantic)	<i>On 1 April the SRSF authorised the SADF to leave their bases Aircraft from all four carriers attacked the Australian base Bombers flying missions to Rabaul and other Japanese bases</i>
k_{2997}^{10}	a “part of” relation (semantic)	<i>In June 2012 she was named as one of the team that competed He was also a part of the Indian delegation Toy Story is also among the top ten in the BFI list of the 50 films you should</i>
k_{2989}^{13}	Ends with a time range (semantic)	<i>Worldwide, most tornadoes occur in the late afternoon, between 3 pm and 7 Weekend tolls are in effect from 7:00 pm Friday until The building is open to the public seven days a week, from 11:00 am to</i>
k_{1935}^{16}	TV shows (semantic)	<i>Time shifting viewing added 57 percent to the episode’s The first season set that the episode was included in was as part of the From the original NBC daytime version , archived</i>

Table 1: Examples of human-identified patterns that trigger different memory keys.

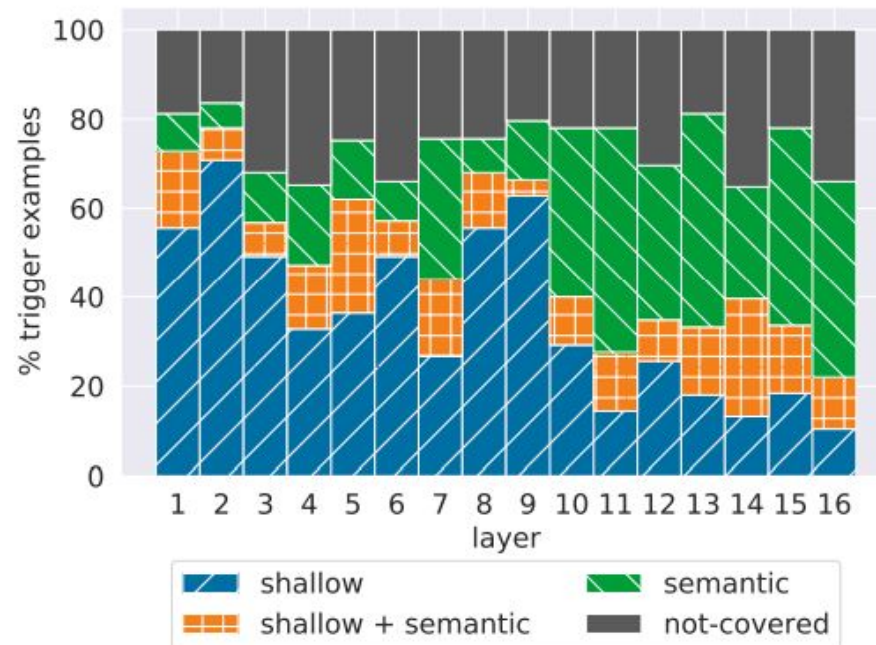


Figure 2: Breakdown of the labels experts assigned to trigger examples in each layer. Some examples were not associated with any pattern (“not-covered”).

From SELF-ATTENTION to FFN

Value	Prediction	Precision@50	Trigger example
$\mathbf{v}_{222}^{15}$	<i>each</i>	68%	<i>But when bees and wasps resemble each</i>
$\mathbf{v}_{752}^{16}$	<i>played</i>	16%	<i>Her first role was in Vijay Lalwani's psychological thriller Karthik Calling Karthik, where Padukone was cast as the supportive girlfriend of a depressed man (played</i>
$\mathbf{v}_{2601}^{13}$	<i>extratropical</i>	4%	<i>Most of the winter precipitation is the result of synoptic scale, low pressure weather systems (large scale storms such as extratropical</i>
$\mathbf{v}_{881}^{15}$	<i>part</i>	92%	<i>Comet served only briefly with the fleet, owing in large part</i>
$\mathbf{v}_{2070}^{16}$	<i>line</i>	84%	<i>Sailing from Lorient in October 1805 with one ship of the line</i>
$\mathbf{v}_{3186}^{12}$	<i>jail</i>	4%	<i>On May 11, 2011, four days after scoring 6 touchdowns for the Slaughter, Grady was sentenced to twenty days in jail</i>

Table 2: Example values, their top prediction, the fraction of their key's top-50 trigger examples that agree with their prediction, and a matching trigger example (with the target token marked in blue).

End APPENDIX